Bakery Hub システム教科書

機能・設計・技術・安全性を理解するための解説書

更新：2026年11月4日。対象：Bakery Hub v3.2実装を含むmain。
基準コミット：6f61e12c5927d31a0be31a5a1e2f89a15b36b859

この本はBakery Hubという店舗運営システムを理解するための資料です。一般的なPython入門書ではありません。画面で行った操作が、どの処理を経てデータとなり、安全に保存・表示されるのかを読み進めます。コードの住所はすべて store/bakery-hub/ を基点とします。

公開状態は上記コミットのREADMEと公開準備記録に基づきます。本書の制作で本番の設定・稼働状態を再取得していません。本番で利用できると記す項目も記録上の状態であり、現在の接続成功を保証するものではありません。資料の説明と公開操作の承認は別です。

目次

第1章｜Bakery Hubの全体像 3

第2章｜店舗運営を支える機能 5

第3章｜画面操作からデータ保存まで 8

第4章｜データ管理と安全設計 10

第5章｜UI/UXと設計思想 12

第6章｜品質管理・テスト 13

第7章｜公開・運用・復旧 14

第8章｜設計の改善と今後の課題 16

付録A｜旧概要から移した実装読解の参照資料 17

第1章｜Bakery Hubの全体像

お店の仕事をつなぐ

想定する利用者は小さなベーカリーの店長とスタッフです。商品を登録し、販売数を記録し、売れ行きを確かめ、翌日の仕込みや材料を考える。さらに出勤予定や店舗のメモも必要になります。Bakery Hubはこれらを店舗ホームから辿れるようにまとめ、紙や別の画面への転記・探し直しを減らすことを目指します。

この目的は「ボタンを増やすこと」ではありません。利用者が今の登録状態と次の操作を把握できることが基準です。業務の最終判断は人が行います。AIの提案も、予報も、販売実績そのものと同じ確実性では扱いません。

システム構成図（概念図）

ブラウザ（HTML・CSS・JavaScript）
↓ HTTPリクエスト：画面を開く／入力を送る
Python・Flask（ルート・権限確認・検証・計算）
↕ SQLAlchemy（モデルを通して問い合わせる）
PostgreSQL（商品・会計・予定などを永続保存）
↑ HTMLまたはJSONの応答を画面へ返す

外部サービスはサーバー側の処理につながります。天気情報・公開RSSは取得して表示し、Gemini APIは必要な分析時に呼び出します。Puoppoは別アプリであり、Bakery HubのDBを共有する分析処理ではありません。

使用技術は仕事の分担

技術	Bakery Hubでの役割
Python / Flask	URLと処理を結び、入力・権限・業務条件を確認する
Jinja / HTML / CSS	データをHTMLへ埋め込み、表示順・余白・押しやすさを整える
JavaScript / Chart.js	入力操作、JSON取得、グラフ、レジの端末側操作を担当する
SQLAlchemy / PostgreSQL	データをモデルとして扱い、制約付きで永続保存する
SQLite	通常のpytestで使う隔離されたメモリ内DB。PostgreSQL固有の保証は別試験
Alembic / Flask-Migrate	DB構造の変更履歴を管理する
Docker / Gunicorn / Render	実行環境を組み立て、Webリクエストを処理・公開する
pytest / Node / Playwright	Python・JavaScript・ブラウザ操作の異なる層を検証する

実装と公開を分けて読む

状態	基準コミットで確認した範囲
本番利用可能と記録	従来のAdmin・Guest日次入力、商品・分析、店舗ホーム等。機能ごとの権限・外部設定条件あり
実装済み・本番未公開	v3.2 Guestの商品・写真・オンライン会計・返金・分析・スタッフ／シフト・仕込み
実装済み・公開判定NO-GO	Admin正式レジ。非保存デモ・監査画面も有効化条件を持つ
開発・検証上の残課題	Guest公開の実機・PWA確認、正式レジ実機、未同期会計のログアウト案内
未対応・今後の課題	Guest廃棄ロス、Guest仕込みへの販売実績参考連携

根拠：README.md、docs/v3.2-guest-launch-readiness.md、docs/v3.2-phase5b-g580-and-rollback.md。11月2日のバックアップ・隔離復元はREADMEに利用者報告として追記されています。古い公開準備記録の「未実行」と同一時点の記述ではありません。

第2章｜店舗運営を支える機能

商品管理：現在の設定と過去の売上を分ける

利用者は商品を登録し、名前・価格・写真・販売状態を管理します。従来の月次Productと、継続して使うRegisterProductは役割が異なります。月次商品は年月とDatasetを持ち、日次数量DailySalesとつながります。レジ商品は会計明細から参照され、名前と単価は会計時点の値も明細に保存されます。後で値上げしても、過去会計の単価を書き換えない設計です。

商品一覧は今月の登録と過去のCatalogを区別します。標準18商品のテンプレートと、Guest v3.2開始時の公開可能な初期商品は同じ意味ではありません。GuestはAdminの実商品を無断で複製しません。写真は許可された素材・アップロード経路で扱い、画像名だけを信用して任意ファイルを読ませません。

担当：app.py、catalog_repository.py、register_product_routes.py、register_products.py、register_images.py、models.py。検証：tests/test_register_product_routes.py、tests/test_register_products.py、tests/test_guest_register_v32.py。入力上限・写真・別Datasetへのアクセス・失敗時のrollbackが確認点です。

レジ・会計・返金：金額の正本をサーバーに置く

写真タイルで商品を選び、数量と預かり金を確認して会計を完了します。返金は販売と別会計として確認します。これは実装済みレジの動作説明であり、正式Admin本番レジの公開済みを意味しません。

ブラウザが送る合計金額を正本にはしません。register_service.pyがDBの商品価格を読み直し、数量から合計・お釣りを計算します。表示時から価格が変わった場合、営業日が変わった場合、旧日次売上が同日にある場合には自動的に保存を押し通しません。返金は負の数量を使い、販売との混在や未確認の返金を拒否します。

担当：register_routes.py、register_service.py、register_products.py、static/register_core.js、static/register_offline_store.jsなど。検証：tests/test_register_routes.py、tests/test_register_foundation.py、tests/test_register_service_postgresql.py、tests_js/register_core.test.cjs、tests_e2e/guest_register_v32.spec.cjs。再送・競合・価格差・期限・保存失敗までが検証対象です。

売上分析：保存した事実を集計する

Dashboardでは期間を選び、売上や数量、商品順位を確認します。ホームの本日売上も同じ利用者のDatasetの集計を使います。記録がない「未入力」と、記録がある合計0円は別の状態です。

旧日次と新レジは単純に足し合わせません。切替情報に基づいて集計元を選び、返金も反映します。会計が端末に残っているだけではサーバー集計へ入りません。v3.2 Guestの分析導線は保存照合後に表示する考え方です。

担当：register_sales_aggregation.py、register_sales_view.py、sales_period_summary.py、app.py、templates/dashboard.html。検証：tests/test_register_sales_aggregation.py、

tests/test_register_dashboard_integration.py、tests/test_register_sales_aggregation_postgresql.py。画面のグラフだけでなく、合計の根拠とDataset境界を確認します。

仕込み・レシピ・廃棄ロス：予定と実績を混同しない

仕込みではレシピを選び、作る数量を入力して必要材料を合算します。production_prep.pyのrequired_ingredients()やcombined_ingredients()が材料量を求めます。入力をwhole_number()やingredient_amount()で検証し、不正な数量を計算へ流しません。レシピ・材料・日付別計画は別モデルで管理します。

廃棄は /prep/waste で日付・商品・数量等を記録するAdmin向け実装です。本番利用可否は今回未確認です。READMEの旧節に「Draft PR・本番未適用」が残る一方、最新の復元報告は後続revisionを示しており、古い説明だけから公開済みとも未適用とも断定しません。waste_records.pyのadmin_dataset()が対象を限定します。仕込んだ予定数だけから「店頭に残っている数」を断定する機能ではありません。Guestの廃棄管理と仕込み販売実績参考連携は未対応です。

担当：production_prep.py、waste_records.py、PrepRecipe・PrepIngredient・PrepPlan・WasteRecord。検証：tests/test_production_prep.py、tests/test_guest_prep_v32.py、tests/test_waste_records.py、tests/test_prep_postgresql.py、tests_e2e/guest_prep.spec.cjs。材料計算と認可を分けて調べ、複数計画の保存失敗で一部だけ残らないことも確認します。

スタッフ・シフト・予定・店舗ツール

スタッフを登録し出勤予定を入力すると、ホームから当日の予定を確認できます。スケジュールは日時のある作業を、メモは共有情報を、タスクは対応状況を表します。用途を分けることで、情報を一つの自由記入欄に詰め込まずに済みます。

担当：shop_staff_shifts.py、shop_schedules.py、shop_memos.py、shop_tasks.py、material_orders.py。モデルはShopStaff・ShopShift・ShopSchedule・ShopMemo・ShopTask・MaterialOrderItem。更新時も認可されたDatasetに絞ります。v3.2 Guestスタッフ／シフトは別の有効化条件を持ち、本番未公開です。

検証：tests/test_shop_staff_shifts.py、tests/test_home_staff.py、tests/test_guest_staff_v32.py、tests/test_guest_staff_hardening.py、tests/test_dataset_authorization.py。存在する他人のIDをURLへ入れた場合も、自分の記録として扱わないことが重要です。

天気・ニュース・AI：情報の由来と限界を表示する

ホームと /weather は地域の予報と対象時刻を表示します。current_weather.pyとhome_forecast.pyが取得・検証・集計を担当します。未提供値を0や推測値で埋めません。DailyWeatherの観測・再解析データとDailyForecastの予報記録は別物です。MET Norwayの取得時刻別データから求めた最大・最小と、全日の日最高・最低も同じ意味とは限りません。

bakery_news.pyは公開RSSの見出し・日付・出典・リンクを取得し、キャッシュや重複除外を行います。本文転載や見出しからの世論断定はしません。Puoppo連携は独立したログイン・分析のサービスへ検索

語を渡す仕組みで、初期は無効、Guestからの引き渡しも無効です。

売上AIはapp.pyとprompts.pyが担当し、許可されたDatasetの集計を基にGeminiへ依頼します。Guestには回数制限があります。外部応答の失敗も業務保存の成功とは分けます。検証では外部サービスをモックし、実Geminiの呼び出しを必要としません。tests_js/home_weather.test.cjsとtests_e2e/home_weather.spec.cjsも表示と欠損時の確認に使います。

第3章｜画面操作からデータ保存まで

現行の日次入力を最後まで追う

GET /input は画面を開くリクエストです。app.pyのinput_sales()が認証を確認し、require_current_dataset()を通じて許可された領域を決め、年月に合う商品と数量を取得してinput.htmlへ渡します。Jinjaは商品を繰り返し配置してHTMLを作ります。

入力後のPOST /inputではrequest.formから日付、getlist()で商品IDと数量の列を受けます。日付の変換、列の長さ、重複ID、数量の範囲、商品のDatasetと年月・販売状態を検証します。ifはここでは「不正な状態なら保存しない」分岐、forは「送られた全商品を調べる」繰り返しです。returnは応答を返して処理を終えるため、不正な行の後に保存処理へ進むことを防ぎます。

検証済みの全行をDBへ送り、同じ商品・日付がある場合は数量を更新します。PostgreSQLとSQLiteではon_conflict_do_update()を使います。commit()でまとめて確定し、DB例外ならrollback()で取り消します。保存成功後はsuccess=Trueを指定して入力画面のHTMLを返します。この処理はDashboardへ自動リダイレクトしません。利用者が分析リンクを開くGET /dashboardは別リクエストです。

Dashboardは集計結果を表示し、/api/dashboard-dataなどのJSONをJavaScriptが使います。request.formは送信された入力、モデルはDBの記録、JinjaはHTML生成、JSONは画面側へ渡すデータ表現です。似た辞書状の形でも責任が違います。

実装済みレジ会計の往復（概念図）

商品選択 → 数量・預かり金の確認 → 同じ会計キーを付けてJSON送信
↓ POST /api/register/transactions
認証・公開条件・Dataset・JSON項目の確認
↓ register_service.save_register_transaction()
入力正規化 → Datasetロック → 既存会計照合 → 価格・営業日確認
↓ 商品と会計明細の計算・保存
1回のcommit → 会計ID等のJSON応答 → 端末側の保存照合・表示

register_routes.pyはJSONの必須項目が過不足なく揃っているか調べます。サービスは認可済みDatasetを受け取ります。入力されたdataset_idだけで権限を決めません。データが不正なら400、権限や公開条件に合わなければ403、価格変更・同じキーの内容違いなどは409、一時的なDB障害は503として返す実装です。これらは同じ「エラー」でも、利用者が取り直すべき入力や再送判断が異なります。

実コードから読む、再送の早期終了

以下はregister_service.pyのsave_register_transaction()からの連続した抜粋です。前後の入力正規化・ロック取得・例外処理は省略しています。

```
    previous = _existing(owner.id, key, digest)
    if previous is not None:
        db.session.commit()
        return previous
```

_existing()はDatasetと会計キーに加え、内容のdigestを照合します。同じ会計があれば、その記録を返して終了します。returnは「会計をもう一つ追加するところへ進まない」ために働きます。違う内容を同じキーで送れば成功として使い回しません。新しいキーを勝手に作って再送すると、この仕組みの前提を崩すので、端末は元のキーを保持します。

JavaScriptは何を担当するか

画面では商品タイルや数量入力、預かり金、通信状態、未同期表示を更新します。JSONを受け取るfetchは通信の窓口で、DBへ直接保存する命令ではありません。画面の「完了」表示だけで保存の正しさを判定せず、サーバーの結果・保存照合を確認します。これが通信切断時に「押せたつもり」を売上にしないための境界です。

第4章｜データ管理と安全設計

モデルのつながり（概要・全テーブルではない）

主体	関連する記録	分離・整合性の要点
Dataset	Product、RegisterProduct、スタッフ、予定等	AdminとGuestごとの領域
Product	DailySales	月次商品と日次数量。商品・日付の重複を制約で管理
RegisterTransaction	RegisterTransactionLine	1会計と複数明細。会計時点の商品名・単価も保持
RegisterProduct	価格履歴、会計明細	現在価格と過去の事実を分ける
PrepRecipe	PrepIngredient、PrepPlan	レシピ定義と日付ごとの計画を分ける
ShopStaff	ShopShift	スタッフと出勤予定。別Datasetの結合を防ぐ

SQLAlchemyのモデル定義には型・外部キー・一意制約・CHECK制約があります。アプリの入力検証に加え、DB自身も不正な状態の保存を拒否します。例えばRegisterTransactionLineは明細金額が単価×数量であることをCHECKし、会計と商品との複合外部キーにdataset_idを含めます。Pythonのフィルターだけを唯一の境界にしません。

認証と認可は別の確認

認証は誰として利用しているか、認可はその人が何に触れてよいかです。Flask-Loginのログイン情報だけで、任意IDの商品へアクセスしてよいわけではありません。require_current_dataset()などで現在の領域を解決し、取得・更新クエリを限定します。Admin・Guest A・Guest Bを別の主体として検証します。

Admin認証にはadmin_sessions.pyとAdminAuthSessionを使います。サーバー側で有効期限と失効状態を都度確認し、ログアウトで失効させます。生トークンはDBへ保存せずhashで照合します。Cookieの存在だけで無期限の認証を成立させません。秘密値や実Cookieは本書にもリポジトリにも掲載しません。

Guestは無操作30分・最長2時間などの制限を持ち、AIは合計3回などの上限があります。期限が切れた瞬間に全行が物理削除される保証とは違います。次のGuest開始等のcleanup時に対象を再確認して削除します。期限・上限を撤廃して使いやすくする設計ではありません。

トランザクションと競合

会計本体だけ保存され、明細が欠けると合計の根拠が壊れます。save_register_transaction()は会計と全明細を一つのトランザクションで確定し、例外ではrollbackします。flush()はIDや制約をDBへ反映する途中操作で、最終的なcommit()と同義ではありません。

同時に二つのリクエストが到着する可能性があります。DatasetロックやPostgreSQLの商品行ロックに加え、UNIQUE(dataset_id, submission_key)が最後の重複防止になります。IntegrityErrorの後はrollbackして、新しいトランザクションで同じ会計を再照合します。すべてのIntegrityErrorを「再送成功」と決めつけない設計です。

オフライン保存と再同期

IndexedDBに会計を残すことは、サーバーへ売上を登録したこととは異なります。レジの端末側保存領域は認証・Datasetのscopeと結びつきます。register_offline_service.pyとregister_offline_store.jsなどが、許可・期限・営業日・価格・再送を扱います。通常のPWAインストールだけで、誰でもオフライン会計ができるわけではありません。

再接続後は同じ会計キーで送信し、サーバーに保存されたか照合します。価格変更や過去営業日など自動保存できないものは復旧確認へ回し、端末の記録を勝手に消しません。Guest v3.2で実装したオンライン会計の公開と、Adminの正式オフライン運用の承認は別です。

未同期会計のあるログアウト時の案内は残課題です。「ログアウトすれば安全に全件同期される」「端末のデータは常に消えない」と断定しないでください。tests/test_register_offline.py、tests/test_register_recovery.py、tests_js/register_offline.test.cjs、tests_e2e/offline_register.spec.cjsで、正常再送だけでなく拒否・復旧・scope境界を検証します。

第5章｜UI/UXと設計思想

状態を見せ、次の操作を伝える

登録済み・未入力・保存できない等を明示し、色に文字とアイコンを添えます。入力に失敗したときは、どこを直せばよいか分かる説明が必要です。危険な操作の意味を利用者の記憶に頼らず、確認・入力制約・サーバー検証を組み合わせます。

ホームの主要カードは空白部分からも目的の一覧へ移動でき、内側の個別リンクはそれぞれの行き先を優先します。CSSで主リンクのクリック領域を広げ、リンクの入れ子やJavaScriptによる二重遷移を避けています。「本日の売上」と「今日の天気」は単なる装飾ではなく、確認作業の入口です。

端末に合わせて操作領域を変える

レジはスマートフォンでは商品2列と縦スクロール、横長タッチ端末では商品2行の横スクロールと右の会計領域、PCでは高さに応じた2〜4行を使います。register_layout.jsは画面の実測寸法と入力方式を見て調整します。数量操作のたびに全商品DOMを作り直さず、位置やフォーカスの安定を守ります。

小さい空白タイルには商品ID・価格・入力を与えず、会計や商品数に数えません。押しやすい寸法、文字が折り返す場合、タブレット回転時にも操作を隠さないことが確認点です。端末差を吸収する処理は、会計の計算処理とは分けてあります。

システム概要の読み方

Web版は5章で基本情報を紹介し、詳しい解説は本書へ分けています。現在の章名を押して目次を開き、章へ移動できます。タッチ端末の条件を満たす画面では、スクロール中にスクラバーが表示され、ドラッグやキーで章を選べます。章数は文書から読み取ります。

JavaScriptが無効でも本文と通常の目次リンクを読めます。JavaScriptは基本情報を隠すためではなく、現在地表示・目次操作・章移動を補助します。動きを減らす設定、見出しへのフォーカス、文字拡大時の見出し位置も確認対象です。

PWAの役割と制限

PWAはホーム画面への追加やキャッシュによる表示を助けます。通常PWAとレジのオフライン専用資産・保存領域は分けて考えます。インストール済みでも通信・認証・期限・公開条件が不要になるわけではありません。本書はmanifest・Service Worker・IndexedDB形式を変更しません。端末の更新やログアウトを含む実機検証は、ブラウザ自動試験とは別に必要です。

第6章｜品質管理・テスト

何を壊さないためのテストか

テスト件数の多さだけでは、事故を検知する力は分かりません。「他GuestのIDを送る」「保存の途中でDBエラー」「同じ会計を同時に送る」など、失敗しやすい条件と守る事実を結びます。修正が入ったら、その不具合が再び起きたときに失敗する検証を残します。

層	主な目的	例
pytest / SQLite	入力・権限・状態・失敗経路を素早く調べる	Dataset越境、期限、rollback
NodeのJavaScript試験	計算や端末状態を小さく検証	会計、数量、再送queue、天気書式
隔離PostgreSQL	DB固有の制約・ロック・競合を調べる	複合FK、同時会計、migration
Chromium E2E	実際のDOMと操作のつながりを確認	スマホ表示、章移動、入力と保存
実機・運用確認	端末・設定・公開段階の条件を確認	PWA更新、全端末の未同期照合

conftest.pyはテスト専用のメモリ内SQLiteを設定し、fixtureで表を準備・片付けます。本番DBにデータがあることを前提にしません。authenticated_client等で認証された主体を用意し、Admin・Guest・匿名それぞれの応答を調べます。時間依存はJSTの営業日とUTCの期限を取り違えないようにします。

システム概要も境界を守る

/system-overviewは公開のGETページです。匿名閲覧で業務DBやAIを呼ばないこと、POSTを許可しないこと、秘密の商品名や任意query値を出さないこと、no-storeやフレーム防御ヘッダーを確認します。説明ページだからといって認可の検証を外しません。

今回の5章化では旧クイズ数・旧9章の期待値を仕様に合わせて変えます。一方、現在地更新、目次の開閉、スクラバードラッグ、キーボード・縮小動作設定・JS無効の基本閲覧は維持すべき保証です。PDFも単に存在するだけでなく、テキストが読めることとレンダリングを確認します。

検証結果と公開判断を分ける

mainに記録された過去のCI合格と、この資料更新で実際に実行した試験は別の証拠です。今回のコマンド・結果・未検証範囲はdocs/system-guide/README.mdに記録します。PostgreSQLや実機を今回実行していなければ、実行済みとして扱いません。

第7章｜公開・運用・復旧

通常起動に保守操作を混ぜない

DockerfileはPython 3.12のマルチステージ構成で依存を組み込みます。通常起動はstartup_guard.pyがrevision・必要table/columnを読み取り確認した後にGunicornを起動します。起動のたびにcreate_all・migration・seedを走らせる設計ではありません。

Renderは記録上、自動デプロイOFFで手動公開を判断します。GitHub mainへ入ったことと本番に反映されたことは違います。公開前には対象コミット、起動設定、DB revision、非秘密の公開フラグ、バックアップと承認を照合します。今回の資料変更にmigrationは不要です。

migrationはデータを守る変更履歴

Alembicのrevisionとdown_revisionが構造変更の順序を表します。モデルファイルを書き換えただけで本番の表が自動で一致するわけではありません。既存の表・外部キー・制約・indexとデータ保全を調べてから変更します。資料制作のためにschemaやrevisionを変える必要はありません。

バックアップは復元確認までを区別する

production_backup_guard.pyと関連手順は接続先の明示確認、暗号化バックアップ、暗号文のSHA-256照合、復号・アーカイブ確認、独立したDBへの復元を扱います。SHA一致はファイルが同じことの確認で、DBが復旧できる証明ではありません。

隔離復元では本番と異なる空の接続先を用意し、復元後にrevision、表・件数、Dataset参照不整合、外部キー孤児、未検証制約、無効indexを確認します。本番の後刻の件数と一致するだけで、取得時点の全データ完全一致とは断定しません。同じsnapshotによる比較がなければ検証の限界を記録します。

READMEにある11月2日のG580検証は利用者報告です。本書制作中に本番backupや実復元を実行したわけではありません。実DB・鍵・パスワード・実データを教材へ添付しません。

商品CSVと全量DBバックアップは用途が違う

data_vault.pyは商品・写真・レシピ・材料等の設定をCSVとManifestへ保存する独立CLIです。空のAdmin Datasetへの復元を前提に、既定dry-run、明示確認後の書き込み、途中失敗時のrollback、保存後の読み戻しを行います。売上・会計・返金・認証・未同期会計は対象外です。DB全量バックアップの代替にはなりません。

公開を止める条件

認可越境、二重会計、集計不整合、cleanupの誤削除、起動や認証の障害があれば先へ進みません。Guest公開とAdmin正式レジ切替を独立して判断します。Guestの公開フラグを変えるためにAdminゲートを解除しません。

旧版へ戻すだけでも、新版で追加されたGuestの仕込み等とcleanupの互換性に問題が出る可能性があります。旧コードへの切り戻しは合成データで確認し、DB自動復元や未同期queueの削除で解決しません。本番変更・merge・deployは別承認です。具体的な操作は既存runbookを参照してください。

第8章｜設計の改善と今後の課題

主な設計変更の理由

月次商品と日次数量の方式から、継続商品・会計本体・明細・価格履歴を持つレジへ設計を広げています。これにより会計時点の価格や返金を扱えます。一方、旧方式と新方式を同日に足すと二重計上になるため、切替情報と旧売上の競合確認が必要になりました。

GuestはAdminに似た流れで商品から会計・分析へ進む設計へ変わりました。ただし、画面を共通にすることはデータ領域や公開条件の共通化ではありません。Dataset分離・期限・容量・AI回数を保ちながら体験を整えます。

システム概要も、長い技術教材と第三者向け紹介が同じ場所にあった状態を見直しました。紹介を5章に整理し、実装を読みたい人向けには本書とコードの住所を残します。理解度8問・Python基礎15問のクイズは撤廃し、解答欄も設けません。

残された課題

v3.2 Guestは本番未公開です。公開条件・実機確認・承認が揃って初めて公開判断ができます。Admin正式レジはNO-GOを維持します。Guest廃棄ロスと仕込みの販売実績参考連携、ログアウト時の未同期会計案内、実機PWA更新等は残課題です。厨房とレジのリアルタイム在庫連携等の将来案を、現在の完成機能として記載しません。

コードと資料の読み順

まずREADME.mdの公開状態を確認し、models.pyで記録の関係を読み、対象ルートとサービス、その失敗を調べるテストへ進みます。行番号は更新でずれるため、ファイル名と関数・クラス名を住所にします。基準版はGitHubの以下のコミットから辿れます。

https://github.com/tosane932/tosane-works/tree/6f61e12c5927d31a0be31a5a1e2f89a15b36b859/store/bakery-hub

関連資料：docs/v3-register-design.md、docs/v3-register-cutover-runbook.md、docs/v3-register-device-test-sheet.md、docs/v3-register-price-approval.md、docs/v3.2-guest-launch-readiness.md、docs/startup-maintenance.md、docs/data-vault-guide.md、docs/g580-restore-verification.md。

本書の生成方法と今回の検証結果：docs/system-guide/README.md。次の付録は旧システム概要から移した実装読解の参照資料です。機能の説明を補う住所として使い、第1章の公開状態と合わせて読んでください。

付録A｜旧概要から移した実装読解の参照資料

旧概要の有用な説明を、機能別の仕組み、実処理の流れ、Pythonの使用箇所、障害の逆引きとして移しました。クイズは移していません。これは基準コミット時点のコード読解資料です。機能の本番公開状態を示す一覧ではありません。標準18商品や月次入力の説明は従来方式のものです。Guest v3.2および正式レジの公開状態は第1・2章を優先してください。

1つの処理を最後まで追ってみる

題材は「売上入力 → DB保存 → Dashboard表示」。今月の登録済み商品に数量を入力する場面を、段階ごとの処理を追います。

ブラウザ操作 → HTTP → Flask / Python → SQLAlchemy → DB → Flask / Jinja → HTML → ブラウザ表示、という往復です。保存とDashboardの表示は、別々のリクエストでつながります。

売上入力画面を開く GET /input で、今月の商品と今日の数量を表示します。 FlaskがURLに対応する input_sales() を呼びます。認証とDatasetを確認し、 _get_current_products() で現在のDataset・今年・今月の販売中商品をqueryします。今日の数量も読み、 render_template() で input.html へ渡します。 Jinjaが商品を繰り返し並べたHTMLを作り、ブラウザがCSSと合わせて表示します。 ここで学べる基礎 Web： GET / route / response Database： query / 絞り込み Frontend： Jinja / HTML / CSS templates/input.html

入力して送信する 「本日の売上個数を更新する」ボタンから、日付・商品ID・数量を送ります。 HTML formの method="POST" により、ブラウザが POST /input を送ります。CSRF tokenの確認と認証を通った後、同じ input_sales() がPOST用の分岐へ進みます。日付は request.form.get("date") 、複数行の商品IDと数量は request.form.getlist() で受け取ります。 ここで学べる基礎 Frontend： HTML form / name Web： POST / request / CSRF Python： if / list

Pythonで入力を確認する 全行を検証し、不正な入力や別Datasetの商品を拒否します。 日付を datetime.date.fromisoformat() で変換し、商品ID・数量の件数一致、重複ID、数量の範囲を確認します。 for で各商品を調べ、認証されたDatasetに属し、売上日と同じ年月の販売中商品であることも確認します。検証済みの行をlistにまとめ、不正なら return で400の応答を返して保存前に終了します。 ここで学べる基礎 Python： if / for / 関数 / list / return Web：validation / 400 Database：Datasetでの絞り込み

DBへ保存する 商品と日付の組み合わせで数量を追加・更新し、まとめて確定します。 DailySales Model のテーブルを使い、商品ID・日付・数量をdictにまとめます。本番用のPostgreSQLと通常pytestの SQLiteでは、 on_conflict_do_update() で同じ商品・日付があれば数量を更新します。 db.session.execute() で全行を保存し、1回の db.session.commit() でtransactionを確定します。DB例外なら rollback() で取り消し、500を返します。 ここで学べる基礎 Python： dict / try / except Database： Model / SQLAlchemy / db.session / transaction / commit / rollback app.py models.py

保存結果からDashboardへ進む 入力画面の成功表示を確認し、「お店の健康診断書（売上分析・ランキング）を見る」リンクを開きます。 保存後は return render_template(..., success=True, ...) で入力画面のHTMLを返します。この処理ではリダイレクトはしません。 return で関数が終わるため、後ろにある

GET用の処理には進みません。利用者がリンクを開くと、ブラウザが新たに GET /dashboard を送り、別の関数 dashboard() が動きます。 ここで学べる基礎 Python： return / 関数の終了 Frontend： Jinja の条件分岐 / HTML link Web：response / 次の GET

Dashboardで表示する 保存済みの数量を集計し、ランキングとグラフに表示します。_get_sales_from_db() がProductとDailySalesをjoinし、Datasetと表示期間で絞ります。数量の合計はDBの db.func.sum() が担当し、Pythonが結果をdictにして sorted() で数量順に並べ、グラフ用のlistも作ります。Flaskが dashboard.html へ渡し、JinjaがランキングのHTMLを、JavaScriptとChart.jsがグラフを表示します。 新しい売上サマリーでは本日・週別・月別・年別を切り替え、 GET /api/sales-period-summary で選択期間の売上金額・前期間比・販売個数ランキング・推移を取得します。今週など進行中の期間では経過日数をそろえて比較します。さらに、店舗所在地を設定した管理者が明示操作した場合に限り、過去の天気データを気象提供元から取得して日付ごとにDBへ保存します。 GET /api/weather-history では保存済みの記録だけを表示し、Guestは天気の取得・保存を実行できません。天気と売上に相関が見えても原因を断定せず、店主の経営判断の材料として扱います。既存の詳細分析は全年度・全月で始まり、期間の「更新」ではJavaScriptの fetch() が GET /api/dashboard-data を送り、Flaskの jsonify() で返されたJSONから画面を更新します。「本日の売上」は日本時間の今日を別に集計します。 ここで学べる基礎 Database： join / query / 集計 Python：dict / list / sorted() Frontend：Jinja / JavaScript / fetch Web：HTML response / JSON templates/dashboard.html

メニュー・機能別の仕組み

役割を確認してから詳細を開くと、Python・DB・画面・テストのつながりを追えます。ファイル名のリンクはGitHubのmainを開きます。

ホーム画面から起動するPWA

Bakery Hub全体をホーム画面へ追加し、アプリのように開くための設定です。インストールと全機能のオフライン対応は別です。

主なURL GET /static/manifest.webmanifest → GET /home → 未ログインなら /login

Python / Flask

PWA（Progressive Web App）は、Web技術で作った画面をホーム画面などから起動する仕組みです。ネイティブアプリのように端末専用の実行ファイルを配布せず、既存のFlask画面をブラウザの仕組みで表示します。起動先の /home は既存の認証を使い、インストールだけではログインやAdmin権限を与えません。HTTPSは通信を暗号化し、インストールなどの前提になる安全な接続です（開発時のlocalhostは例外）。

Database

DB構造・Dataset境界・Cookieは変更しません。Manifestやアイコンに認証情報を含めません。未同期会計が入るIndexedDBと、画面資産を保存するCache Storageは別の保存先です。今回どちらも初期化・削除しません。

Frontend

Web App Manifestは、アプリ名・アイコン・起動URL・アプリ内として扱うURL範囲（scope）・表示方法を伝えるJSONです。共通 _pwa_head.html をログイン・主要画面・レジに含め、同じアプリとして扱います。 standalone は通常のURLバーやタブを省く表示で、Androidのシステム操作を隠すfullscreenではありません。画面の縦横方向を固定せず、戻る操作やリンクも既存ブラウザに任せます。

Service Workerは画面の裏で通信を扱えるJavaScriptで、Manifestとは役割が異なります。今回は新しいService Workerは登録しません。既存のレジ専用Workerだけが /register/ を担当し、Cache Storageへ情報を含まない専用シェルと許可された資産を保存します。ホーム・認証画面・売上APIをオフライン表示するキャッシュは追加しません。未準備端末では通信断後にホームからレジへ進める保証もありません。準備済みレジの既存URLからの復帰とPIN・端末許可・再同期を維持します。

pytest

公開Manifestと画像サイズ、主要画面の共通設定、認証・ログアウト、HTMLのno-storeを確認します。Chromium E2EでManifestの解析、遷移・戻る・更新、Worker登録範囲、通信断時の非キャッシュ、未同期会計保持を確認します。自動テストは実Androidへのインストール成功を意味しません。svitoo P10・Androidスマホ・iOSは実機確認が必要です。

確認するコード

static/manifest.webmanifest static/pwa/icon-source.svg templates/_pwa_head.html app.py static/register_offline_sw.js tests/test_pwa.py tests_e2e/pwa.spec.cjs README.md

共通サイドメニューはホーム・販売業務・店舗運営・情報収集と学習の4群です。Adminの「仕込み管理」には「製造予定・材料管理」（ /prep ）と「廃棄・ロス管理」（ /prep/waste ）の2つの入口があり、計画と実績を別々に操作します。商品登録は商品一覧から進み、共通管理が無効な環境には旧月次登録の入口を残します。Guest専用フラグが有効なら共通商品一覧を利用できます。日次入力とレジモードは既存の公開状態・切替日で出し分けます。Admin専用のレジ管理・検証はフラグに従って表示し、初めは閉じています。Jinjaの表示条件は入口の整理であり、各URLの認可やDataset境界に置き換わるものではありません。

店舗運営ホーム

パン工房 ひだまりの今日の予定・予報・売上を確認します。Adminは出勤予定も見られます。

主なURL GET /home・GET /weather・GET /api/home/weather・GET /bakery-news

Python / Flask

home() が認証済みDatasetの当日予定を返します。Adminだけは business_today() のJST業務日を使い、planned_work_for_day() で同日の出勤・終日予定を読みます。GETはスタッフ情報を変更しません。現在の天気は設定済みの地点について別APIで取得し、失敗時は理由を表示します。Open-Meteoの現在モデル推定と、MET Norwayの直近予報を設定で選択できます。MET NorwayのExpiresまでキャッシュし、過去天気の確定記録を現在の天気として流用しません。正常なMET Norway予報の取得後はCSRF付きPOSTがサーバー側の予報を再確認し、Datasetごとに日本時間の当日最初の予報だけを DailyForecast へ保存します。これは観測結果ではありません。ホームを開かなかった日は保存されず、保存失敗時もホー

ム画面は閲覧できます。

当日詳細・最大7日間の予報は同じ取得レスポンスから home_forecast.py で検証します。MET Norway の時刻別気温はJSTで集計し、全24時間が揃わない日は「取得分・時間帯限定」と表示します。日本向けcompactの降水確率は未提供で、降水量を代用しません。Open-Meteoは既存の利用設定を維持し、日別気温と日内最大の降水確率だけを使用します。数値欠損は推測しません。MET Norwayの十分な朝・午後の時刻別予報がそれぞれ単一の区分で揃う場合に限って「のち」を表示し、Open-Meteoの日別コードだけからは変化を推測しません。週間予報は一時的な表示データで、既存の予報記録POSTや過去天気の保存対象を増やしません。

Database

店舗の予定は現在のDatasetの ShopSchedule だけを読みます。出勤予定カードは許可されたDatasetの ShopStaff ・ ShopShift を再利用します。従来Guestはスタッフ検索を行いませんが、v3.2 Guest専用ゲート有効時は自分のDatasetの予定だけを表示します（本番未公開）。予定の取得失敗時はrollbackして取得不可と示し、0人とは表示しません。当日売上はダッシュボードと同じ集計関数を使い、 DailySales の行の有無で未入力と0円を区別します。天気は GET /api/home/weather で気象庁の設定済み南部／北部の短期予報と広島県全域の週間予報を別々の区域名で表示でき、6時間帯の降水確率を期間付きで示します。気象庁モードには明示的な区域コードが必要で、予報を履歴DBへ保存しません。既存MET Norway モードの DailyForecast は取得当時の予報を保存し、 DailyWeather の過去の気象データと同じ日でも別々に表示します。DBの一意制約とON CONFLICTで同日の再保存・上書きを防ぎ、Guest期限切れ時は削除します。保存済み記録の閲覧では外部APIを呼びません。Render無料DB切り替え時はデータ移行が必要です。

Frontend

store_home.js が日本時間の時計と現在・次の予定を更新し、1分ごとに当日の店舗予定を再取得します。バナーと見出しにはモデル店舗名を表示し、Bakery Hubのブランド名も維持します。スマホでは予定と天気を2列、スケジュールと売上を全幅で示します。Adminの出勤予定カードは売上の下に4人までをコンパクトに示し、人数超過時は残り人数とシフト表へのリンクを表示します。終日は時刻未指定、既存の翌日終了はその旨を明示します。勤務予定から実際の出勤・勤務中を断定しません。ホーム以外では親メニューからホームへ移動し、ホーム上では同じ行をボタンとして子メニューを開閉します。

主なホームカードは既存の主リンクの範囲をCSSで広げ、カード内の予定登録・スタッフ登録・記事リンクを優先します。リンクの入れ子や二重遷移を避け、TabとEnterで操作できます。ホームのPuoppo単独カードを除き、商品一覧・日次入力または公開済みレジ・店舗メモへは幅全体を押せるボタン型リンクで進みます。天気カード全体は専用の GET /weather へのリンクです。 home_weather.js がJST日時や欠損値を整形し、 weather_detail.js が週間予報をtextContentで表示します。

bakery_news.py は農林水産省・食品新聞のパン分野・食品産業新聞社の公開RSSから、関連する最近30日以内の見出し・公開日時・出典・記事リンクを選び、情報源別に30分キャッシュします。失敗した情報源を分離し、古い記事や同じURLは除外します。ホームのJavaScriptは別APIを非同期で読み込むため、ニュース取得失敗はホーム閲覧を妨げません。AI分析や本文の転載は行いません。

「ニュース・AI分析」ではBakeryNewsの記事一覧とPuoppoへの入口を同じページにまとめます。ホームはBakeryNewsカードを入口とし、Puoppo単独カードは表示しません。Puoppoカードは独立したアプリへ検索語をGETで引き渡す準備です。初期状態では無効で、Guestは利用できません。Puoppoの認証・許可メール・回数制限を確認して有効化した後も、Bakery HubではGeminiを呼びません。分析はPuoppo側の明示操作でニュース見出しを元に行います。

pytest

未認証アクセス、CSRF、Dataset分離、出勤と終日・休みの区別、空状態・取得失敗・HTMLエスケープ・閲覧時に更新しないこと、当日売上の未入力と0円、同時保存の一意制約、日時の境界、Guest cleanup、migrationの往復、予報と観測記録の区別をpytestで確認します。

確認するコード

app.py shop_staff_shifts.py current_weather.py home_forecast.py static/home_weather.js tests/test_home_forecast.py forecast_history.py weather_history.py models.py templates/store_home.html templates/weather_detail.html static/weather_detail.js bakery_news.py static/style.css static/store_home.js tests/test_home_staff.py tests/test_forecast_history.py tests/test_store_operations.py

スケジュール

店舗全体の予定を日付・時刻順に確認し、登録・詳細・編集・削除できます。

主なURL GET /schedule・POST /schedule/new・POST /schedule/<id>/edit・POST /schedule/<id>/delete

Python / Flask

shop_schedules.py のBlueprintがフォームの日時と文字数を検証し、日本時間からUTCへ変換します。時刻が過ぎても予定を自動完了しません。Taskのリマインダー機能はまだありません。

Database

ShopSchedule はDatasetに所属し、別DatasetのIDを指定しても取得できません。保存失敗時はrollbackし、Guest期限切れ時には既存cleanupで関連予定を削除します。新しい表はAlembicのmigrationで追加します。

Frontend

Jinjaの一覧・詳細・フォームから予定を操作します。保存・削除はCSRF付きPOST、ホームはJavaScriptで予定を更新します。Taskは既存の店舗メモツールが管理します。

pytest

CRUD、無効な日時、CSRF、Admin・Guest間の越境拒否、DB例外とGuest cleanupを確認します。

確認するコード

shop_schedules.py models.py migrations/versions/d9e4b1c7a630_add_shop_schedules.py templates/schedule_list.html tests/test_store_operations.py

スタッフ・シフト管理

管理者がスタッフを登録し、日別の出勤・終日・休み予定を月間・日別で確認します。勤怠実績は記録しません。

主なURL GET /staff-shifts・GET /staff-shifts/staff/<id>・GET /staff-shifts/shifts/<id>・POST /staff-shifts/shifts/new

Python / Flask

shop_staff_shifts.py のBlueprintがGETで月間・日別一覧とスタッフ・勤務予定の詳細画面を返します。詳細から編集や確認画面へ進み、CSRF付きPOSTで登録・更新・在籍状態変更・予定の取り消しを行います。 admin_required でGuestと未認証利用者を拒み、認証済みAdmin DatasetのIDで詳細IDも含めて全行を絞ります。スタッフ名ではなくIDで特定します。

Database

ShopStaff は表示名・担当・任意の入社日・在籍状態を、 ShopShift はJSTの業務日と出勤・終日・休みの区分を保存します。通常の出勤はUTCの開始・終了日時を保持し、終日・休みは時刻をNULLにするCHECK制約で区別します。Datasetを含む複合外部キーが別DatasetのスタッフIDを拒否し、スタッフと業務日の一意制約が二重登録を防ぎます。退職は無効化で履歴を保持し、DB例外ではrollbackします。ShopSchedule とProduct・DailySalesは変更しません。

日時と画面

入力した日本時間を Asia/Tokyo からUTCへ変換して保存し、表示時にJSTへ戻します。新しい時刻付き勤務は同じ日の最大12時間までとし、同一スタッフの時間帯重複を拒否します。既存の22:00〜翌02:00などの日付またぎ・12時間超のデータは自動変更せず、時刻を変えない備考編集で元の日時を維持します。終日は時刻未指定で24時間勤務ではありません。PCは月間カレンダー、スマホは短い日付選択と日別一覧を示し、一覧行から詳細へ進みます。 planned_work_for_day() は指定JST業務日の出勤と終日をAdminホームにも返し、勤怠実績は判断しません。

Frontend

Jinjaの一覧・詳細・フォームはスタッフ名と備考を自動エスケープします。 shop_staff_shifts.js は終日・休みの時刻欄を隠し、スマホで選択中の日付を見やすくします。サーバー側の入力検証は常に適用します。

pytest

詳細IDを含むAdmin専用認可、入力・CSRF・XSS、退職後の履歴、12時間境界、終日と休み・未登録の区別、日付またぎデータ保全、時刻重複、DB一意制約とDataset外部キー、rollback、migrationのupgrade・downgradeをpytestで検証します。

確認するコード

shop_staff_shifts.py models.py migrations/versions/c8e6a1b9d4f2_add_shop_staff_shifts.py migrations/versions/d5a9f1b7c204_add_all_day_shop_shifts.py templates/shop_staff_shifts.html templates/shop_staff_detail.html templates/shop_shift_detail.html templates/shop_shift_form.html tests/test_shop_staff_shifts.py tests/test_shop_all_day_migration.py

仕込み管理

管理者が「製造予定・材料管理」で日別の予定を立て、「廃棄・ロス管理」で実際の製造・廃棄数を独立して記録します。

主なURL GET /prep・GET /prep/recipes・POST /prep/recipes/new・POST /prep/plans/bulk・POST /prep/plans・GET/POST /prep/waste

試験環境の参考表示

統合集計を明示的に有効化したテスト・ローカル開発環境では、 prep_sales_reference.py が対象営業日の確定済み販売数を読取専用で表示します。旧商品IDと標準テンプレートキーを区別し、返金を差し引き、未記録と0個も区別します。独自レジ商品は名前だけでレシピへ結び付けません。本番の既定設定では非表示で、予定数や材料に自動反映しません。

Python / Flask

production_prep.py のBlueprintがGET /prepでJSTの今日の仕込み、GET /prep/recipesでレシピ管理を表示します。①パンを選ぶ、②個数を決める、③材料を見る順に操作し、CSRF付きPOST /prep/plans/bulkで選択した商品の予定をまとめて保存します。選択を外すだけでは削除せず、0個も予定として保持します。 admin_required と require_current_dataset() でAdmin Datasetを確認し、ID指定時もDatasetで絞ります。日付・全ID・全個数を検証し、Dataset行のロックと一意制約で並行POSTを扱い、一度のSQLAlchemy transactionでcommitします。失敗時は全件rollbackします。

Database

PrepRecipe は月次Productと独立したレシピです。標準・承認済みテンプレートは安定した商品キー、自由入力の月次商品はProduct IDを元の識別子とし、商品名一致で統合しません。 PrepIngredient はレシピの材料と単位・Decimal量、 PrepPlan はJST対象日・レシピID・予定個数を保存します。Datasetを含む複合外部キーと一意制約で別Datasetへの関連付けと二重登録を拒否します。レシピは翌月も再利用できます。過去日の必要量は現在のレシピで再試算し、過去の製造実績を表しません。

実製造・廃棄

waste_records.py はAdminだけがJSTの営業日と継続商品IDまたは旧月次商品IDを選んで実製造数・実廃棄数・メモを保存・修正します。 WasteRecord はDataset、営業日、どちらか一方の商品ID、保存時の商品名を持ちます。実製造数か廃棄数の一方だけでも記録でき、数量のNULLは未入力、0は確認済みの0個です。同じ日の同じ商品は一件のみで、商品とDatasetを複合外部キーで照合します。両方入力して廃棄数が実製造数を超える場合は明示的な確認なしでは保存せず、同じ内容の修正ではDBへ書き込みません。保存済み記録は折りたたんで表示し、保存結果を該当商品付近に示します。仕込み予定、販売、商品

マスタは書き換えず、廃棄数を自動算出しません。CSRF付きPOSTで検証後commitし、DB失敗時はrollbackします。

DB構造と公開

migration c8d3e7f1a204 は親revision b6d2e9f4a107 の後に waste_records 表と商品参照用の制約を追加します。 startup_guard.py は起動前にAlembicのheadとDBのrevision、必要な表・列を読み取り専用で照合し、不一致なら起動を拒否します。PRのマージやテスト成功だけでは本番DBは更新されません。migrationの適用は通常起動と分けた承認済み保守作業です。

Frontend

Pythonの Decimal で 使用量 × 製造予定数 ÷ 基準個数 を計算し、小数第6位まで不足しない方向で表示します。材料名の前後空白だけを揃え、名前と単位の両方が同じものを合算します。
production_prep.js は±10ボタンと1個単位の数値直接入力で同じ再計算を呼び、BigIntの固定小数点で画面を再読み込みせず試算します。変更中は未保存と示し、保存済みの結果と区別します。商品選択で表示を切り替えます。内訳と在庫比較は必要時だけ開きます。在庫は一時入力で、開閉・再計算中も入力値を保持し、未入力と0を区別して不足・余裕・未確認を表示します。在庫減算や自動発注はしません。既存の材料発注リストへ移れます。発注メモの明示的な電話行は妥当な国内番号だけ tel: リンクにします。

pytest

tests/test_production_prep.py が3ステップ表示・一括保存のatomic性とrollback・配合・端数・材料の区別・日付別の更新・Admin認可・Dataset分離・CSRF・エスケープ・発注電話を確認し、tests/test_prep_migration.py が新テーブルのupgrade・downgradeと既存商品保全を確認します。

実績テスト

tests/test_waste_records.py は未入力と0、日別修正、認可・CSRF・rollbackを、tests/test_waste_migration.py と tests/test_waste_postgresql.py はmigrationと複合外部キーを確認します。

確認するコード

production_prep.py waste_records.py prep_sales_reference.py register_sales_aggregation.py models.py migrations/versions/e2a7c4d9b105_add_prep_recipes_plans.py migrations/versions/c8d3e7f1a204_add_waste_records.py templates/prep_index.html templates/prep_recipe_form.html templates/prep_plans.html templates/waste_records.html static/production_prep.js static/waste_records.js material_orders.py tests/test_production_prep.py tests/test_waste_records.py tests/test_waste_migration.py tests/test_waste_postgresql.py startup_guard.py docs/startup-maintenance.md tests/test_register_image_prep_reference.py tests/test_prep_postgresql.py tests/test_prep_migration.py

旧月次の商品・メニュー登録

18品の標準テンプレートとAdminが申請した商品カタログから選び、月ごとの商品と売価を登録します。自由入力もできます。

主なURL GET /・POST /・POST /admin/product-catalog

試験環境のレジ商品画像

通常の月次商品登録とは別に、試験用の /register/products では承認済みカタログの画像をレジ商品へ選択できます。 register_images.py がGuestへの公開設定と画像ファイルを確認し、標準18品は軽量WebPへ解決します。画像変更は商品ID・名前・価格・過去の会計明細を変えません。商品管理用の閲覧・書込フラグは本番既定で無効です。Adminの商品管理フラグ有効時は商品一覧へ統合し、新しい写真も共通商品へ登録できます。この月次登録の説明は従来Guestと旧入力用です。v3.2 Guestは共通商品管理を使います（本番未公開）。

Python / Flask

index() が request.args から表示月、 request.form.getlist() から複数行を取得。 if で年月・文字数・売価（0〜9,990円、10円単位）・商品IDをサーバーでも検証し、 for で更新内容を組み立てます。テンプレートの初期価格は入力開始値なので、登録前に売価を変更できます。テンプレート名は固定で、「新メニューを追加」なら自由入力できます。

Database

カタログはテンプレートの元データで、標準18品は product_images.py と static/product_images/ 、Admin追加分は product_catalog/admin_products.json と static/product_images/admin/ にあります。月次 Product は別にPostgreSQLで管理し、カタログへの追加だけでは作成されません。Product.image_key は画像を識別し、差し替え後も旧画像の識別子をJSONに残します。 DailySales は Productに紐付きます。月次登録ではDatasetと年月で絞り、外した商品は is_active=False とし、DBエラーならrollbackします。

Frontend

index.html のJinjaがテンプレートを並べ、JavaScriptが行の追加・削除、年月切替、商品名・代表画像・初期価格の設定を担当します。画像なしテンプレートは「画像なし」で登録できます。Adminだけの「商品カタログへ追加」フォームは multipart/form-data で商品名・初期価格・ひらがなの読み・任意画像を送ります。inline SVGの画像選択ボタンから選び、後から追加・差し替えも可能です。Guestには編集UIを出さず、サーバーも403で拒否します。

GitHubへの保存

Adminの申請は request.files でJPEG・PNG・WebPを受け、実画像・MIME・入力10MiB上限を検証し、品質を保って必要なら縮小してからWebPへ再符号化します。保存画像は500KiB以下です。元ファイル名は保存名に使わずUUIDにします。GitHub APIでJSONと画像を1つのGit tree・commitにまとめ、専用branchからレビュー用PRを作ります。PR作成に失敗した場合はそのbranchを削除します。mainへは直接書かず、承認・mergeと次のDeploy後にテンプレートへ反映します。GitHub用secretの本番設定は別途必要です。

pytest

月次登録の名前・価格・ID、Dataset越境、画像key、履歴維持とDB rollbackに加え、Admin申請の認可、画像なし、入力・画像検証、JSONと画像の同一commit、PR失敗時のbranch削除を確認します。GitHub APIはmockで検証します。

確認するコード

app.py models.py product_images.py product_catalog/admin_products.json catalog_repository.py templates/index.html tests/test_admin_product_catalog.py tests/test_catalog_repository.py tests/test_products.py

Phase 4のスタッフ体験は既定OFFの独立ゲートで制御します。認証主体のDatasetでスタッフとシフトを照合し、期限切れ削除ではシフトをスタッフより先に削除します。pytestではGuestの期限切れHTTP拒否とAdmin/Guest越境を確認し、隔離PostgreSQLではロック競合とrollback後のスタッフ・シフト保持も検証します。画面のリンク表示とサーバーの認可は同じ条件を使用します。

Guest公開はコード更新・フラグ有効化・実機確認を別工程で判定します。Guest仕込みは既定OFFの専用ゲートで公開し、register:商品IDとDatasetでレシピを識別します。共通のDecimal計算と一括保存を使い、期限切れ削除はリンク・予定・材料・レシピを商品より先に削除します。製造予定は在庫や販売実績へ自動反映しません。

v3.2のGuestレジは専用フラグ（既定OFF）で公開します。新規Guest開始時に公開可能な標準商品と切替日を同じDBトランザクションで保存し、失敗時はまとめて取り消します。共通の商品編集・写真登録・会計保存を利用し、認証主体のDatasetだけを読み書きします。UUIDによる再送照合、商品30件の独自枠、会計1,000件の上限、期限とCSRF保護を維持します。切替日以降の旧日次入力はPOSTでも拒否し、ダッシュボードは共通の排他集計を利用します。Adminの本番認可とオフライン端末許可は開放しません。

v3.2 Phase 3では、Guestの会計完了後に register.js が端末内の直前の会計キーと同期結果を照合し、サーバー保存を確認した場合だけ「売上分析を見る」を表示します。端末への記録だけでは売上確定と案内しません。ダッシュボードは既存のDataset別の期間集計を使い、販売・返金を差し引きます。AIは別のPOST操作で対象年月の売上を利用し、Guest共通の利用回数制限を維持します。JavaScript・pytest・Chromium E2Eで境界を検証します。

商品一覧・共通商品マスタ

Adminの商品管理フラグ有効時は写真・価格・分類・販売状態を一か所で管理し、レジが同じ商品を読み込みます。Guest専用フラグ有効時も同じ画面を使い、無効な環境では月次Catalogを維持します。

主なURL GET /products・GET/POST /register/products/new・POST /register/products/<id>/status

Python / Flask

product_catalog() が require_current_dataset() で利用者のDatasetを確認します。フラグ有効のAdminは共通商品一覧へ分岐し、Guestと旧環境では business_today() でAsia/Tokyo基準の年月を決めて月次Catalogを返します。いずれのGETも商品を生成しません。

Database

Guestと旧環境では、そのDatasetの Product だけを年・月・IDの降順で取得。今月の is_active=True を現在商品にし、過去月のProductから同名の最新履歴を選びます。今月inactiveでも前月の履歴を確認できます。 set で同名の重複を避けます。GET時にProduct・DailySalesは変更しません。

Frontend

Guestと旧環境は products.html のJinjaで「今月登録の商品」と「過去に登録された商品」を分け、画像・商品名・最新価格・登録年月／最終登録年月を表示します。過去Productの is_active=True を現在の「登録中」とは表示しません。過去のProductと画像識別子は保持します。カタログへ追加しただけの商品は、月次Productを作るまでここには出ません。

共通商品マスタ・写真中心の登録

RegisterProduct を唯一の継続商品マスタとして使います。Adminは商品一覧の「＋ 新しい商品を登録」から承認済み画像・アップロード・スマホ撮影、商品名、0〜100,000円の整数価格、パンの種類8種類、定番・季節もの・特別商品を確認して保存します。 validate_image() は画像の実体・MIME・容量を検証し、メタデータを除いた500KiB以下のWebPへ変換します。写真はDBの photo_webp へ保存し、認可済みDatasetの写真だけをURLで返します。Renderの一時ファイルへ保存しません。

終売・再販売・価格履歴

is_active=False の商品は一覧下部へ残り、新規会計から外れます。再開時は価格を確認し、同じID・写真・分類・レシピ対応を再利用します。新しい販売状態カラムや物理削除は追加しません。送信キーは写真と分類も含めて二重登録を検査し、更新は署名したDataset・ID・ revision とDatasetロックで競合を検出します。価格変更は RegisterPriceHistory へ追記し、既存明細のスナップショットを変更しません。初期18品も同じ商品で、確認付きPOSTだけが候補を追加します。Adminの管理画面・レジのGETでは生成しません。

スマホは2列・縦、タブレットは2行・横、PCは高さに応じた行数

register_pager.js は slice() で最大12商品ずつの内部グループを保ち、タブレットではCSS Gridの列方向へ正方形タイルをタッチ主体なら2行、マウス主体なら高さに応じ2〜4行で配置します。
overflow-x:auto だけで自然に横へ動き、ページ先頭への吸着や見えている隣グループの操作禁止はありません。写真は object-fit:contain で全体と縦横比を保ち、名前は2行です。独立した＋は置かず、既存 .add-one をタイル全体へ広げ、写真・名前・価格・余白から1個追加します。−と数量入力は追加ボタンと兄弟の独立要素にし、ボタンを入れ子にしません。−は28pxの見た目と48×48pxのタッチ領域を分け、数量・価格と同じ下段へ整理して写真を拡大します。スマホ幅では全商品を一つの2列グリッドに並べ、縦スクロールで探します。横移動・カテゴリ分割・未登録枠は置きません。タブレットでは最後の縦1列の不足分だけを薄いグレーの未登録枠で補います（2行なら最大1枠）。商品0件では空状態の案内を優先します。空き枠には商品ID・操作・入力がなく、aria-hiddenで読み上げからも除き、商品数や会計には含めません。タブレットの前次ボタン・位置表示は商品見出しの横へまとめ、左右キーも使えます。回転では同じ実商品DOMを移すため、数量・カート・預りを保持します。pointerの移動でスワイプ後の誤追加を防ぎ、会計状態には触れません。

預りキーパッドと固定会計

register_cash_pad.js は0〜9・00・C・末尾1桁削除を提供し、桁区切りの円表示を更新します。通常のテキスト入力やソフトキーボードは不要です。キーパッド内でPCの数字・Backspace・Delete/Cも使えます。先頭ゼロを正規化し、上限超過は元入力を保持して知らせます。既存のinputイベントを発行するため、 RegisterCore.calculation() のお釣り計算と下書き保存を再利用できます。タブレットの右側は明細だけをスクロールさせ、合計・預り・お釣り・完了が見える範囲を確保します。空き明細を引き伸ばさず、実際の明細量だけを合計の上に置きます。スマホの下部固定「会計へ」は合計とともに更新され、キーパッド直上へ移動します。会計操作中は固定ボタンを隠し、「商品一覧へ戻る」で元の縦位置へ戻れます。register_layout.jsはinnerHeightとvisualViewport.heightの小さい方からヘッダー・警告・商品見出しの実測高を引き、残り高さ÷行数と商品領域の幅でタイルの一辺を決めます。最低168pxまたは文字サイズの10.5倍を保ち、伸縮した明細の高さを必要高の根拠にしません。横長画面のテンキーは数字3列の右に削除・クリア・00を置く4段で、キー52〜60px、完了56pxを確保します。商品なしの重複案内は省き、商品ありでは最低28pxまたは先頭明細の実測高を確保します。会計部を除いた残り高さを明細の上限にし、そこだけをスクロールさせます。横長画面はヘッダー36px、戻る・前次の見た目32px／タッチ領域44pxです。長文説明帯はバッジへ集約しますが、練習用・未保存、開発用・未公開や保存停止、同期状態・重要通知を残します。合計・お釣り前の罫線と余白を省き、お預かりは金額と同じ行にします。空明細の固定44pxや固定12pxのスクロール余裕に頼らず、実測して収まる場合に1画面にします。低い画面・文字拡大・長い警告では縦スクロールへ戻し、重要な操作を隠しません。端末設定はヘッダーから開くダイアログ、保存失敗や要確認は画面上部に残します。

過去売上・仕込み・オフラインの保全

旧 Product ・ DailySales は履歴として保持し、同名でも自動統合しません。旧商品からのコピーは明示確認した別IDで、売上はコピーしません。既存の仕込み対応は同じ商品IDと RegisterPrepLink のままです。オフライン準備では写真も既存の暗号化商品カタログに含め、未同期会計の価格・明細・送信キー・営業日は変えません。署名した商品版の照合・再送防止・要確認も維持します。非保存デモは同じ横スクロールとキーパッドで練習し、DB・IndexedDBへ会計を保存しません。正式レジ本番切替はNO-GOのままです。

pytest

固定した10月の日付で今月・過去の区分、inactive時の過去履歴、最新価格と年月、同名重複なし、Dataset分離、画像なし、GETで商品と売上が変わらないことを確認します。

確認するコード

app.py models.py register_product_routes.py register_products.py register_images.py static/register_pager.js static/register_layout.js static/register_cash_pad.js static/register.js templates/register.html templates/register_products.html tests/test_unified_products.py tests/test_product_catalog.py

日次売上入力

商品ごとの販売数量を日付別に保存し、同じ商品・日付の入力は数量を更新します。

主なURL GET /input・POST /input

Python / Flask

input_sales() が日付を datetime.date.fromisoformat() で変換。商品と数量の件数、数量の範囲、商品が属する年月・Dataset・販売状態を全行で検証してから保存します。レジ会計への切替日以降は、POST側でDatasetをロックして旧日次売上の保存を拒否します。

Database

DailySales.product_id で Product につながります。商品IDと日付には一意制約があり、PostgreSQL / SQLiteの on_conflict_do_update() で重複行を増やさず更新します。複数行の保存は1回のcommitで確定します。

Frontend

input.html が登録済み数量を表示し、HTML formで送信。JavaScriptは数量欄のフォーカス時の選択や「今日のひとこと」の取得を担当します。

開発中のレジ

写真タッチ式のレジは本番の既定設定で表示も保存も無効です。開いた画面では通信断でも計算とIndexedDBへの原子的な会計記録後に次の会計へ進めます。サーバー発行の商品情報と独立した会計控え・現金日報を確認した案件だけ、管理者が元の送信キーと営業日で例外的に確定できます。端末時計だけを根拠にせず、営業日や価格の不一致は要確認として保持します。短期の端末許可、PIN暗号化した商品版、専用Service Workerのシェルから再起動後に復帰する経路と、承認済み翌営業日の自動同期を追加しました。端末時計は証拠ではなく、実端末検証・本番切替は未実施です。

Admin用のレジ操作デモ

独立した REGISTER_V3_PUBLIC_DEMO_ENABLED を有効にした場合だけ、Adminが /register/demo で標準18品の写真タッチ、数量、お預かりとお釣り、返金確認を練習できます。 register_demo.js は正式レジと同じ register_core.js で計算し、練習会計の完了を画面に表示します。商品と売上のDB書込、IndexedDBの会計キュー、正式会計APIの通信を行いません。画面を閉じると入力は消え、通常の売上集計にも含まれません。Guestはアクセスできません。

Phase 5Bの切替準備（通常公開では無効）

REGISTER_V3_READINESS_ENABLED が明示的に有効な場合だけ、Adminが /register/readiness で自分のDatasetの旧新売上、切替境界、会計明細、販売可能商品の不足・100,000円超の旧商品を読取専用で監査し、予定日を保存せずに検証できます。サーバーは端末のIndexedDB内の件数を把握できません。/prep/register-links では商品IDとレシピIDを確認して独自商品を一対一で対応付けます。複合外部キーとDatasetロックで越境・重複を防ぎ、既存レシピキーと製造予定は変更しません。新migrationの本番適用とフラグ有効化は今回行いません。

開発中の保存状況確認

認証済みDatasetの会計キーをGET照会し、端末記録と確定会計の明細・単価・合計が一致した場合だけ「保存済み」にします。POSTの成功応答だけでは同期済みにしません。不一致は要確認、通信失敗と該

当なしは未確認のまま保持します。Adminの元記録には送信キー・申告営業日・端末申告時刻を表示します。端末時刻は証拠ではありません。保存を緊急停止しても本番の照合GETは利用できます。デモでは使用しません。

Phase 5Cの切替・価格履歴

導入後のAdmin商品価格を register_price_history へ追記し、導入以前を推測しません。register_cutover_service.py はDataset行をロックして旧新の重複・明細不一致を再監査し、承認記録と切替日を同じtransactionで保存します。切替後の旧日次売上POSTと新会計POSTも同じロックを使います。migrationとフラグの本番適用、切替操作はPhase 5Dへ分離しています。

未同期会計の復旧案件（本番既定無効）

register_recovery_service.py はAdminが端末の元会計を売上と別の案件へ保全し、同じ送信キーの再登録とDataset越境を拒否します。元明細と確認履歴は追記専用です。未発見は未保存の証明ではなく、保存済みは元記録との完全一致で判断します。調査中・保留・取消の判断と証拠参照を記録し、案件のある会計は通常の自動再送で売上にしません。独立した日付つき控え・現金日報と署名された商品情報を確認したときに限り、Datasetロック下で元のキーと営業日に確定し、会計・明細・承認監査を同時保存します。切替監査は未解決案件を止めます。別経路の事前承認済みオフライン同期は、Datasetロックと元キー、商品版、短期許可、旧日次売上との排他を検査します。範囲外の会計だけ復旧案件へ回し、正式公開の前に実端末と本番データを検証します。

pytest

追加と更新、別日付の保持、数量の境界、重複ID・別月・販売終了商品の拒否、複数行の不正入力やDB障害で一部だけ保存されないことを確認します。PostgreSQL用には同じ商品・日付への並行送信テストもあります。

確認するコード

app.py models.py templates/input.html tests/test_sales.py tests/test_sales_postgresql.py

売上データ分析 / Dashboard

本日・週・月・年の売上金額推移と前期間比・売れ筋ランキングを表示し、従来の販売数量分析と日本時間の本日の売上も利用できます。

主なURL GET /dashboard・GET /api/dashboard-data

Python / Flask

_get_dashboard_period() で表示期間をそろえ、 _get_sales_from_db() の辞書を sorted() で数量順にします。グラフ用の名前と数量はlistに分け、HTMLまたは jsonify() で返します。

Database

現在の本番既定設定では Product と DailySales をjoinします。開発環境または将来のAdmin本番用の独立フラグを有効にした場合は、切替日前の旧行と当日以降の確定会計を排他的に読み、返金を差し引きま

す。PostgreSQLでは読み取り専用の同一スナップショットを使い、集計異常は売上数値として表示しません。

Frontend

dashboard.html のJavaScriptが fetch() でJSONを取得。Chart.jsの売上推移と独立したランキングカードを更新し、年月指定の分析も別セクションから使えます。スマホのランキングだけ約3.5件の高さで内部スクロールし、端ではページスクロールに戻ります。PCでは全順位を表示します。日付単位のグラフだけに保存済み天気のアイコンを重ね、選択すると予報と過去気象データの両方の詳細を確認できます。履歴を開く際に気象APIは呼びません。AIボタンは別のPOSTを送ります。

pytest

HTMLとAPIの期間一致、年・月の不正値、数量とグラフの一致、販売終了商品の履歴、他Datasetの除外、本日の金額の期間独立性を確認します。

確認するコード

app.py templates/dashboard.html tests/test_dashboard.py tests/test_business_date.py

店舗メモツール / 材料発注

材料名・数量のメモ・補足をチェックリストに登録し、完了・未完了を切り替え、不要な項目を削除します。外部への発注送信は行いません。

主なURL GET /material-orders・POST /material-orders など

Python / Flask

material_orders.py のBlueprintが処理をまとめています。 _validate_create_form() で文字数などを確認し、完了操作では「切り替える」ではなく希望する状態を受け取ります。

Database

MaterialOrderItem をDatasetごとに保存。 is_completed と completed_at を合わせて更新し、未完了を先に表示します。DB障害時は変更をrollbackします。

Frontend

material_orders.html のHTML formで追加・完了・削除を送信。JavaScriptは追加ボタンから入力欄へフォーカスを移します。削除前の確認も用意しています。メモの「電話：」行から検証済みの番号だけを tel: リンクにし、タップするまで発信しません。

pytest

入力の正規化・境界、件数上限、並び順、同じ完了状態の再送、他Datasetの操作拒否、CSRF、不正入力やDB障害でデータが変わらないことを確認します。

確認するコード

material_orders.py models.py templates/material_orders.html tests/test_material_orders.py tests/test_production_prep.py

店舗メモツール / 店舗メモ

メモの作成・編集・検索・固定・複製と、ゴミ箱への移動・復元・完全削除を行います。スマホ編集では本文を自動保存します。

主なURL GET /shop-tools/memo・POST /shop-tools/memo/autosave など

Python / Flask

shop_memos.py のBlueprintがformとJSONを処理。本文からタイトルを作る関数、文字数の検証、検索文字の扱い、削除状態ごとの条件分岐を用意しています。

Database

ShopMemo に本文と日時を保存。 deleted_at があるものをゴミ箱、 pinned_at があるものを固定として扱います。Datasetで絞り、commit / rollbackで変更を確定・取消します。

Frontend

Jinjaのメモ一覧・ゴミ箱と shop_memos.js が担当。入力イベントと fetch() による自動保存、操作メニュー、取り消し表示を実装。通常一覧のURLは文字として表示し、ゴミ箱では検証したHTTP(S) URLをリンクにします。

pytest

CRUD、検索、固定の順番、複製、ゴミ箱と復元、自動保存、文字数・件数、HTMLのエスケープ、Dataset越境、失敗時rollbackを確認します。

確認するコード

shop_memos.py templates/shop_tools/memos.html templates/shop_tools/memo_trash.html static/shop_memos.js tests/test_shop_memo_routes.py

店舗メモツール / タスク

タスクの追加・編集・完了・お気に入り・並び替えと、選択したタスクや完了済みタスクの削除を行います。

主なURL GET /shop-tools/tasks・POST /shop-tools/tasks/reorder など

Python / Flask

shop_tasks.py のBlueprintがform / JSONを取得。タイトル、真偽値、並び替えIDのlistを検証し、重複・欠落・別DatasetのIDを拒否します。

Database

ShopTask の position で順番、 is_starred でお気に入りを保存。完了状態と完了日時を一緒に更新し、並び替えや一括削除もまとめてcommitします。

Frontend

shop_tasks.html と shop_tasks.js が担当。click・pointer・touchイベントで完了、編集、長押し選択・並び替えを扱い、JSONで保存します。完了操作の取り消しと送信中のボタン制御も実装しています。

pytest

タイトルの境界、CRUD、完了の再送、お気に入り、保存される並び順、一括削除、別Datasetの拒否、CSRF、DB障害時rollbackを確認します。ジェスチャーの見た目や触り心地はブラウザでも確認する必要があります。

確認するコード

shop_tasks.py templates/shop_tasks.html static/shop_tasks.js tests/test_shop_tasks.py

Guest Demo

ログイン画面からデモを開始し、ゲスト専用のデータ領域で各機能を試せます。管理者や別ゲストのデータとは分離しています。

主なURL POST /guest/start

Python / Flask

start_guest_session() で専用Datasetと GuestUser を作成し、Flask-Loginで認証。 load_user() で復元・期限確認し、 require_current_dataset() で利用可能な領域を決めます。

Database

Dataset が期限・活動日時・AI利用回数を持ちます。Guest作成回数と同時利用数を制限し、期限切れのデータをcleanupします。各機能のqueryは認証されたDatasetに限定します。

Frontend

login.html のCSRF付きformで開始。共通メニューは「ゲストデモ」と表示し、終了ボタンで認証Sessionを終了します。

pytest

開始、期限、偽装した認証情報、Dataset分離、作成制限・同時利用数、cleanupが管理者や有効なGuestを消さないこと、AI利用上限を確認します。

確認するコード

app.py models.py templates/login.html tests/test_guest_entry.py tests/test_guest_identity.py tests/test_guest_cleanup.py tests/test_dataset_authorization.py

管理者ログイン・ログアウト

設定された単一管理者を認証します。業務ページは管理者または正規Guestが利用でき、システム概要は未ログインでも読めます。

主なURL GET /login・POST /login・POST /logout

Python / Flask

check_password_hash() で照合し、Admin専用のログイン記録をDBへ保存してから login_user() で認証します。 load_user() はパスワード設定とログイン記録の有効性を毎回確認し、発行から12時間の絶対期限・失効済み・不明な記録を拒否します。DB障害時も認証を許可しません。保護されたrouteのデコレータがアクセスを制御します。

Database

管理者の認証設定は環境設定で読み、ユーザーの登録テーブルはありません。 AdminAuthSession はログインごとの識別子のSHA-256と発行・期限・失効時刻だけを持ち、生の識別子やCookieは保存しません。ログアウトは該当するログインだけを失効させ、別端末の独立したログインは維持します。オフライン有効時は既存どおり店舗の端末許可も失効させ、この2つを同じtransactionで確定します。業務データとGuestのDataset・期限は変更しません。

Frontend

JinjaとHTML formで認証情報をPOST。ログアウトもCSRF付きPOSTです。FlaskのSession Cookieを使って認証状態を引き継ぎます。

pytest

成功・失敗、旧方式Cookieの再ログイン要求、ログアウト後のコピーCookie拒否、別端末の認証維持、12時間の期限境界、DB障害時の拒否、端末許可との一括rollback、migrationの業務データ保持、POST限定とCSRFを確認します。

確認するコード

app.py config.py admin_sessions.py models.py templates/login.html tests/test_auth.py tests/test_authorization.py tests/test_admin_login_rate_limit.py tests/test_logout.py tests/test_admin_session_revocation.py tests/test_admin_session_migration.py

AIアドバイス・今日のひとこと

売上分析では販売数量から改善案を生成し、売上入力では短いひとことを表示します。どちらもボタン操作で取得します。

主なURL POST /api/ai-advice・POST /api/greeting

Python / Flask

google.genai でGeminiを呼び、 prompts.py で売上用の文章を組み立てます。15秒のtimeoutを指定し、 try / except で失敗を扱い、JSONで結果を返します。

Database

売上分析の入力は現在のDataset・適用した期間の数量集計です。GuestのAI利用回数はDatasetで管理し、上限を超える呼び出しを防ぎます。AIの生成結果を売上レコードとして保存する処理ではありません。

Frontend

JavaScriptがCSRF token付きの fetch() を送信。待機表示とボタンの無効化を行い、結果は textContent で表示します。

pytest

Geminiクライアントを置き換え、期間・Datasetが正しい入力、timeout指定、API失敗時の代替メッセージ、CSRF、Guest利用上限を確認します。実APIの生成品質を保証するテストではありません。

確認するコード

app.py prompts.py templates/dashboard.html templates/input.html tests/test_ai_integration.py tests/test_ai_public_defense.py tests/test_guest_ai_limit.py

Python基礎・応用マップ

用語だけ覚えるより、該当する機能を開き「この処理を変えると何が起きるか」を考えてからコードを読んでみましょう。

Python基礎・応用マップ ：値 → 条件 → 繰り返し → 関数とデータ、の順に読むと処理を追いやすくなります。気になるカテゴリだけ開き、教材の住所からGitHubのmainへ進めます。

実装済みの使用例だけを掲載しています。業務処理・テスト・初期データ作成・migrationのどこにある例かを明示します。読める範囲の地図であり、Python全仕様を網羅する一覧ではありません。

値・型・基本演算

入力文字、価格、数量、状態を、値と型から読み解きます。

変数・代入

これは何か

変数は値に付ける名前です。=は右側の結果を左側の名前へ代入し、==による比較とは区別します。

Bakery Hubでは

商品登録のindex()は、name.strip()の結果をnormalized_nameへ代入します。 商品・メニュー登録の仕組み

なぜ使うか

受け取った入力と、検証・保存に使う整えた名前を区別して読めます。

教材の住所

業務処理： app.py → index

実コードの一行 normalized_name = name.strip()

str / int / bool・型変換

これは何か

strは文字列、intは整数、boolはTrue / Falseの状態。型変換は値を別の型として扱える形にすることです。

Bakery Hubでは

_parse_bounded_nonnegative_integer()は数字の文字列を検証してintへ変換。タスクの_parse_boolean()はboolまたは文字列の0 / 1を状態へ変換します。 日次売上入力の仕組み

なぜ使うか

文字列の数量をそのまま計算せず、受け付けた値を整数や状態として扱います。変換できたことだけで入力が妥当とは限りません。

教材の住所

業務処理： app.py → _parse_bounded_nonnegative_integer 業務処理： shop_tasks.py → _parse_boolean

実コードの一行 return int(normalized_value)

比較・算術演算子・真偽値

これは何か

< / <= / == / !=は比較、+ / - / * / %などは計算です。比較の結果や空でない値を条件判断に使います。

Bakery Hubでは

index()は商品名の長さを範囲比較し、価格を%で割った余りから10円単位か判断します。 商品・メニュー登録の仕組み

なぜ使うか

価格の大小だけでなく、許可した単位かも保存前に確認できます。%は割り算の余りです。

教材の住所

業務処理： app.py → index

条件分岐

条件ごとに処理を選び、不正な値や権限のない操作を止めます。

if / elif / else・and / or / not

これは何か

ifは条件が成り立つ場合、elifは別の条件、elseはそれ以外を選びます。and / or / notは条件を組み合わせます。

Bakery Hubでは

require_current_dataset()は管理者、Guest、それ以外を分け、管理者領域またはそのGuestの領域だけを解決します。 Guest Demoの仕組み

なぜ使うか

認証済みというだけで全利用者のデータを返さず、操作してよいDatasetを確定するための分岐です。

教材の住所

業務処理： app.py → require_current_dataset

is / is not・Noneの判定

これは何か

isは同じオブジェクトかを確認します。値の一致を確認する==とは役割が違い、Noneの有無はis None / is not Noneで調べます。

Bakery Hubでは

index()は既存商品を見つけられなければexisting_product is Noneで拒否。入力された商品IDがあるかも別に検証します。 商品・メニュー登録の仕組み

なぜ使うか

見つからない商品を別の商品として更新しないためです。Noneと数量0・空文字は同じ状態ではありません。

教材の住所

業務処理： app.py → index

繰り返し

全行の確認と、条件に応じて続ける処理を区別します。

for

これは何か

forは反復できるデータから要素を取り出し、順番に処理します。

Bakery Hubでは

input_sales()は商品IDと数量の各組を確認し、検証済みの売上をまとめて保存します。 日次売上入力の仕組み

なぜ使うか

1行目だけでなく全商品を検証し、不正な行を含む送信を保存前に拒否するためです。

教材の住所

業務処理： app.py → input_sales

実コードの一行 for product_id, quantity in zip(product_ids, quantities):

while / break / continue

これは何か

whileは条件が成り立つ間の繰り返し。continueは次の周回へ進み、breakは繰り返しを終えます。

Bakery Hubでは

_split_url_suffix()はURL候補の末尾の句読点や余分な閉じ括弧を1文字ずつ取り分け、不要な末尾がなくなればbreakします。 店舗メモ・ゴミ箱の仕組み

なぜ使うか

メモの文末記号をURLの一部にせず、リンクと後ろの文章を分けます。このリンク化処理はゴミ箱の表示に使われます。

教材の住所

業務処理： shop_memos.py → _split_url_suffix

実コードの一行 url = url[:-1]

データ構造

並べる、対応させる、重複を除く、複数の値を返すための入れ物です。

list / dict・get() / items()

これは何か

listは順番を持つ要素の集まり。dictはキーと値の対応です。get()はキーから値を取得し、items()はキーと値の組を取り出します。

Bakery Hubでは

input_sales()は検証済み行をlistへ。dashboard()は商品名と数量のdictをitems()で取り出して並べ替えます。 売上データ分析の仕組み

なぜ使うか

複数行の順序と商品名から数量への対応を、それぞれに適した構造で表せます。dict.get()とFlaskのrequest.args.get()は同じライブラリの機能ではありません。

教材の住所

業務処理： app.py → input_sales 業務処理： app.py → dashboard

set・重複除去・in / not in

これは何か

setは同じ値を重複して持たない集合です。in / not inは要素が含まれるかを確認します。

Bakery Hubでは

input_sales()のseen_product_idsで同じ商品IDの二重送信を拒否。商品一覧のproduct_catalog()でもsetで同名履歴の重複を避けます。 日次売上入力の仕組み

なぜ使うか

同じ売上を送信内で二度扱うことと、一覧の重複表示を防ぎます。目的により同名判定とID判定を使い分けます。

教材の住所

業務処理： app.py → input_sales 業務処理： app.py → product_catalog

実コードの一行 seen_product_ids = set()

tuple・アンパック

これは何か

tupleは複数の値をひと組にします。アンパックは、その組を複数の名前へ分けて受け取る書き方です。

Bakery Hubでは

_validate_title()は整えたタイトル、元の入力、エラーをまとめて返します。create_task()は3つの名前へ受け取ります。 タスクの仕組み

なぜ使うか

検証結果と入力欄へ戻す値を一度の関数呼び出しで渡せます。何番目が何を表すか、返す側と受け取る側を見比べます。

教材の住所

業務処理： shop_tasks.py → _validate_title 業務処理： shop_tasks.py → create_shop_tasks_blueprint.create_task

関数

入力から戻り値まで、処理のまとまりを追います。

def・引数・return・デフォルト引数・helper

これは何か

defで処理を定義し、引数で必要な値を受け取り、returnで結果を返します。デフォルト引数は省略時の値です。helperは処理を分けた補助関数の呼び方です。

Bakery Hubでは

dashboard()から_get_sales_from_db(current_dataset, target_year=None, target_month=None, *, row_limit=None)を呼び、集計dictを受け取ります。*の後の引数は名前付きで渡します。 売上データ分析の仕組み

なぜ使うか

画面処理とDB取得を分け、対象領域・期間・必要な上限を呼び出し側から明示できます。returnはその関数の処理を終了します。

教材の住所

業務処理： app.py → _get_sales_from_db 業務処理： app.py → dashboard

型ヒント

これは何か

引数や戻り値がどの型を想定するかを読み手や解析ツールへ示す注釈です。注釈を書くだけで実行時の型チェックは行われません。

Bakery Hubでは

prompts.pyのbuild_sales_prompt(sales_summary: str) -> strは、販売数量の説明文字列を受け取り、AIへ渡す文字列を返します。 AIアドバイスの仕組み

なぜ使うか

入力と出力の意図を読みやすくします。現在のアプリ全体に広く導入されているわけではなく、入力検証の代わりにもなりません。

教材の住所

業務処理： prompts.py → build_sales_prompt

実コードの一行 def build_sales_prompt(sales_summary: str) -> str:

クラス・オブジェクト

インスタンスの属性と、継承して使う機能を確認します。

class・instance・__init__・self・属性・継承・property

これは何か

classから作る個々のオブジェクトがインスタンス。__init__で初期化し、selfからそのインスタンスの属性へアクセスします。継承は既存classの仕組みを引き継ぐことです。

Bakery Hubでは

AdminUserとGuestUserはFlask-LoginのUserMixinを継承。GuestUser.__init__()がdataset_idを持たせ、@propertyのidがguest:で始まる識別子を返します。 ログイン・Guest認証の仕組み

なぜ使うか

Guestごとに領域を持たせ、認証へ渡す識別子を属性のように取得します。propertyは保存済みのDB列を作る機能ではありません。

教材の住所

業務処理： app.py → AdminUser 業務処理： app.py → GuestUser 業務処理： app.py → GuestUser.id

PythonのclassとSQLAlchemy Model

これは何か

Pythonのclassを基礎に、SQLAlchemyがテーブル・列・関連付け・DBへの読み書きの仕組みを組み立てています。普通のclassを定義するだけでDBへ保存されるわけではありません。

Bakery Hubでは

models.pyのProduct(db.Model)とDailySales(db.Model)は、商品と日別実績の列・関係を定義しています。 商品登録・売上保存の仕組み

なぜ使うか

商品のオブジェクトを扱うPythonと、永続化するDBの構造をつなぎます。db.sessionへの追加やcommitと、単なる属性の代入を区別します。

教材の住所

業務処理： models.py → Product 業務処理： models.py → DailySales

例外処理

失敗の種類を分け、半端な保存や後片付け漏れを防ぎます。

try / except・rollback

これは何か

tryで処理を試し、指定した例外が起きるとexceptで対応します。rollbackはPython構文ではなく、SQLAlchemyの変更取消処理です。

Bakery Hubでは

input_sales()は日付・商品IDの変換失敗を拒否し、DB保存中のSQLAlchemyErrorではdb.session.rollback()して失敗を返します。 日次売上入力の仕組み

なぜ使うか

エラー応答だけを返して一部の売上を残すことを避けます。Pythonの例外処理がDBのtransactionを守る呼び出しにつながります。

教材の住所

業務処理： app.py → input_sales

raise・独自例外・例外のつながり

これは何か

raiseは例外を発生させます。独自classで失敗の種類を定義できます。raise ... from errorは元の原因と新しい例外を結び付けます。

Bakery Hubでは

商品画像のvalidate_image()はInvalidCatalogImageを発生させ、通信処理はCatalogPublishErrorを使います。GuestCapacityFullは満員と他の503障害を分けます。 商品カタログ申請の仕組み

なぜ使うか

画像の形式不正、通信失敗、Guest満員を文字列比較だけで分類せず、それぞれの対応へ進むためです。

教材の住所

業務処理： catalog_repository.py → validate_image 業務処理： catalog_repository.py → InvalidCatalogImage 業務処理： catalog_repository.py → CatalogPublishError 業務処理： app.py → GuestCapacityFull

再raise・finally・migrationの検証

これは何か

except内で引数なしのraiseを使うと元の例外を再送出します。finallyはtryの終了時に実行する後片付けです。

Bakery Hubでは

seed_demo_data()はrollback後に再raiseし、失敗を成功に見せません。Dataset移行の_validate_backfill()は件数や所有者を確認し、不整合ならRuntimeErrorを発生させます。 Datasetの分離と移行の仕組み

なぜ使うか

初期データ作成やDB移行の失敗を呼び出し側へ伝え、変更前後のデータ保全を確認します。ロックを解放するfinallyの例は「with / yield」で辿れます。

教材の住所

初期データ作成： seed_demo.py → seed_demo_data migration： migrations/versions/c7a1d9e4f2b6_create_datasets_and_backfill_products.py → _validate_backfill

Pythonらしい反復・データ処理

番号・組・並び順・条件判定を短く表す書き方です。

enumerate() / zip()

これは何か

enumerate()は番号と要素を取り出し、既定では0から番号を付けます。zip()は複数の並びから同じ位置の要素を組にします。

Bakery Hubでは

reorder_tasks()は受信したID順にpositionを保存。input_sales()は商品IDと数量をzip()で対応付けます。 タスク並び替えの仕組み

なぜ使うか

タスクの順番と商品ごとの数量を明示できます。zip()は短い方に合わせて止まるため、売上処理は先に件数一致を検証しています。

教材の住所

業務処理： shop_tasks.py → create_shop_tasks_blueprint.reorder_tasks 業務処理： app.py → input_sales

実コードの一行 for position, task_id in enumerate(task_ids):

sorted() / lambda / key / reverse

これは何か

sorted()は新しい並びを作ります。keyは比較に使う値を取り出す関数、lambdaは短い無名関数、reverse=Trueは降順の指定です。

Bakery Hubでは

dashboard()は商品名・数量の組について、item[1]の数量をkeyにして多い順へ並べます。 売上ランキングの仕組み

なぜ使うか

商品名順ではなく販売数量順のランキングを作ります。SQLの集計とPythonの表示順の整理は別の担当です。

教材の住所

業務処理： app.py → dashboard

実コードの一行 ranked_sales = sorted(sales_data.items(), key=lambda item: item[1], reverse=True)

内包表記・any()・ジェネレータ式

これは何か

内包表記は反復からlist / dict / setを組み立てます。any()は1つでも真ならTrue。丸括弧などで書くジェネレータ式は必要に応じて値を取り出します。

Bakery Hubでは

タスクrender_list()は内包表記で完了・未完了のlistへ分け、_parse_task_ids()はany()とジェネレータ式で不正なIDがあるか調べます。 タスク一覧とID検証の仕組み

なぜ使うか

表示対象の整理と不正値の検出を表せます。list内包表記とジェネレータ式は同じ入れ物ではなく、後者はlistを先に作りません。

教材の住所

業務処理： shop_tasks.py → create_shop_tasks_blueprint.render_list 業務処理： shop_tasks.py → _parse_task_ids

all() / range()はテスト側で読む

これは何か

all()はすべての要素が真かを確認し、空ならTrueです。range()は一定範囲の整数を反復できます。

Bakery Hubでは

ログイン上限のテストはrange()で試行を繰り返します。PostgreSQLテストのall()は匿名キーの長さ・文字と複数行をまとめて検証します。 ログイン試行回数のテストの仕組み

なぜ使うか

上限まで試した結果や保存形式を確認するためのテスト用例です。SQLAlchemyのquery.all()は行を取得するメソッドで、Python組み込みのall()とは別です。

教材の住所

テスト： tests/test_admin_login_rate_limit.py → test_five_failed_logins_are_allowed_and_sixth_is_rate_limited テスト： tests/test_admin_login_rate_limit_postgresql.py → test_concurrent_failed_admin_logins_never_exceed_postgresql_limit

文字列処理

入力を整え、長さや行を確認し、表示する文章を組み立てます。

strip() / splitlines() / len() / スライス

これは何か

strip()は両端の空白、splitlines()は行の分割、len()は長さの確認。スライスは一部分を取り出す書き方です。

Bakery Hubでは

_derive_title_from_body()は最初の空でない行を探し、[:SHOP_MEMO_TITLE_MAX_LENGTH]でタイトルを切り出します。 店舗メモの自動保存の仕組み

なぜ使うか

スマホで本文を自動保存したとき、先頭の空行をタイトルにせず、上限内の見出しを作ります。

教材の住所

業務処理： shop_memos.py → _derive_title_from_body

実コードの一行 stripped_line = line.strip()

join() / f文字列・casefold()

これは何か

str.join()は複数の文字列を区切り付きで連結し、f文字列は値を文章へ埋め込みます。casefold()は大文字・小文字差の比較に使う変換です。

Bakery Hubでは

_generate_ai_advice()は商品名と数量の文章をjoin()でまとめます。publish_catalog_change()はcasefold()でカタログの同名登録を確認します。 AIに渡す売上説明の仕組み

なぜ使うか

AIへ渡す説明の整形と、文字の大小だけが違う重複登録の防止に使います。SQLAlchemyのquery.join()によるテーブル結合とは別です。

教材の住所

業務処理： app.py → _generate_ai_advice 業務処理： catalog_repository.py → publish_catalog_change

型と状態の確認

値がない、False、数量0を区別し、JSONの想定外の型を拒否します。

isinstance()・boolとintの区別

これは何か

isinstance()は値が指定した型のインスタンスかを確認します。Pythonではboolもintの一種として判定されるため、用途によって別に除外します。

Bakery Hubでは

_parse_task_ids()はJSONがdictか、task_idsがlistか、各IDが正のintかを確認し、True / FalseはIDとして受け付けません。 タスク操作のJSON検証の仕組み

なぜ使うか

数値IDのつもりで状態値を使われることを防ぎます。型だけでなく、正数・重複・所有者も別の条件で検証します。

教材の住所

業務処理： shop_tasks.py → _parse_task_ids

None / is None / is not None

これは何か

Noneは値がないことを表します。Falseは偽の状態、0は数値であり、意味を区別して扱います。

Bakery Hubでは

memo_json_payload()はpinned_at is not Noneから固定状態を作ります。ShopMemoは削除日時deleted_atを持ち、ShopTaskはis_completedとcompleted_atを対応させます。 メモの固定・ゴミ箱の仕組み

なぜ使うか

固定していない、ゴミ箱に入っている、完了しているなどの状態を、日時の有無とboolで明確に表します。

教材の住所

業務処理： shop_memos.py → create_shop_memos_blueprint.memo_json_payload 業務処理： models.py → ShopMemo 業務処理： models.py → ShopTask

モジュール・import

標準ライブラリ、外部ライブラリ、自作モジュールの役割を分けます。

import / from ... import ...

これは何か

モジュールはコードをまとめた単位。importは他のモジュールの機能を使い、from ... import ...は名前を指定して取り込みます。

Bakery Hubでは

app.pyはPython標準のdatetimeやhashlib、外部のFlaskやSQLAlchemy、自作のmodels・prompts・shop_tasks等を読み込みます。 AIアドバイスの組み立ての仕組み

なぜ使うか

すべてを自作せず、日付、HTTP、DB、業務処理を担当ごとに組み合わせます。_generate_ai_advice()から自作build_sales_prompt()へも処理を辿れます。

教材の住所

業務処理： app.py → _generate_ai_advice 業務処理： prompts.py → build_sales_prompt

標準ライブラリの道具

ここで扱う標準ライブラリはPythonに含まれる道具です。Flask等の外部ライブラリと区別します。

uuid / json / io

これは何か

uuidは識別子、jsonはデータの読書き、ioはメモリ上の読み書き用バッファなどを提供します。

Bakery Hubでは

Datasetはuuid.uuid4で識別子を作り、load_admin_catalog()はjson.loadで商品設定を読みます。validate_image()はio.BytesIOで画像の入力・再符号化結果を扱います。 商品カタログ・画像の仕組み

なぜ使うか

Guestを区別し、商品設定を読むことと画像データを処理することを、それぞれの道具で表します。画像の解析自体は外部ライブラリPillowです。

教材の住所

業務処理： models.py → Dataset 業務処理： product_images.py → load_admin_catalog 業務処理： catalog_repository.py → validate_image

hashlib / hmac / ipaddress

これは何か

hashlibはハッシュ計算、hmacは秘密鍵を使う照合やメッセージ認証、ipaddressはIPアドレスの解析に使います。

Bakery Hubでは

_get_admin_auth_fingerprint()は認証設定の指紋を作り、load_user()はhmac.compare_digestで照合。_get_rate_limit_client_key()はIPを正規化しHMACの匿名キーへ変換します。 認証状態と試行上限の仕組み

なぜ使うか

認証設定の変更を検知し、試行上限をclientごとに管理します。ハッシュと暗号化は同じ処理ではなく、IPや秘密値をこのページに表示しません。

教材の住所

業務処理： app.py → _get_admin_auth_fingerprint 業務処理： app.py → load_user 業務処理： app.py → _get_rate_limit_client_key

functools / contextlib / threading

これは何か

wrapsは包んだ関数の名前等を引き継ぎ、contextmanagerはwith用の処理を作ります。threadingは同一プロセス内の同期等を扱います。

Bakery Hubでは

admin_or_guest_required()はwrapsを使用。_serialize_admin_login_attempt()はSQLiteではthreading.Lock、PostgreSQLではDBのadvisory lockを使用します。 ログインと同時試行の仕組み

なぜ使うか

認証デコレータの関数情報を保ち、同時のログイン試行を整理します。threading.Lockだけで別プロセスまで守れるという説明にはしません。

教材の住所

業務処理： app.py → admin_or_guest_required 業務処理： app.py → _serialize_admin_login_attempt

正規表現

文字列の形を調べます。型・パス・URLの検証とは組み合わせて使います。

re・compile() / finditer() / fullmatch()

これは何か

正規表現は文字列の形をパターンで表す道具。finditer()は一致箇所を順に取り出し、fullmatch()は文字列全体が一致するか確認します。

Bakery Hubでは

_linkify_memo_body()はHTTP(S) URL候補を探します。商品画像のavailable_product_images()と_safe_image_path()は画像キーやAdmin画像名をパターンで確認します。 メモ・ゴミ箱のURL表示の仕組み

なぜ使うか

メモのURL候補と、承認済み画像の識別子を確認します。URL候補はさらにurlsplitで検証し、画像はさらに配置範囲・実ファイルも確認します。

教材の住所

業務処理： shop_memos.py → _linkify_memo_body 業務処理： product_images.py → available_product_images 業務処理： product_images.py → _safe_image_path

pathlib・ファイルとパス

ファイルの場所を表すPathと、ファイルを開く処理を確認します。

pathlib.Path・ファイルの読み取り

これは何か

パスはファイルやディレクトリの場所。Pathなら/での結合、name / suffix、resolve()、is_file()等を使え、文字列の連結と違って操作をまとめて表せます。

Bakery Hubでは

_safe_image_path()はproduct_images配下の実ファイルかを確認し、load_admin_catalog()はwithとencoding="utf-8"でJSONを開きます。 商品画像と商品設定の仕組み

なぜ使うか

入力名をそのまま画像の場所に使わず、承認したディレクトリの内側にあることを確かめます。Pathを使うだけで安全になるわけではありません。

教材の住所

業務処理： product_images.py → _safe_image_path 業務処理： product_images.py → load_admin_catalog

context manager・with・yield

準備 → 一時的に使う → 片付ける、という流れを追います。

with / @contextmanager / yield / finally

これは何か

withは利用前後の処理をcontext managerへ任せます。@contextmanagerではyieldの位置で呼び出し側へ処理を渡し、戻ってから片付けます。

Bakery Hubでは

_serialize_admin_login_attempt()はロック取得後にyieldし、finallyでrollback・SQLiteのロック解放を行います。login()がwithで利用します。 管理者ログインの仕組み

なぜ使うか

認証処理の途中に例外があっても片付けるためです。このyieldは大量データの配信ではなく、ロックを使う期間を囲む用途です。

教材の住所

業務処理： app.py → _serialize_admin_login_attempt 業務処理： app.py → login

pytest fixtureのyield

これは何か

pytestのyield fixtureは、yieldまでが準備、渡した値をテストが使い、yieldの後が片付けです。pytestがその流れを管理します。

Bakery Hubでは

conftest.pyのflask_app()は隔離したメモリ内SQLiteにテーブルを作り、アプリをyieldし、テスト後にsessionを片付けてテーブルを削除します。 売上入力などのpytestの仕組み

なぜ使うか

テストごとのデータを分けるための処理です。アプリ側のcontext managerと同じyield構文でも、呼び出して管理する仕組みと目的は異なります。

教材の住所

テスト： conftest.py → flask_app

デコレータ

@の行が何を登録・追加しているのかを確認します。

route・Blueprint・認証デコレータ

これは何か

デコレータは関数等に別の処理を適用する書き方です。何をするかはデコレータごとに異なり、@があるだけで認証されるわけではありません。

Bakery Hubでは

appの@app.route等はURLを登録し、Blueprintの@blueprint.get / postは店舗ツールのURLを登録。@admin_or_guest_requiredや@access_requiredが認証・認可を追加します。 タスク画面の取得の仕組み

なぜ使うか

URLと処理を結び、関数本体へ進む前に利用者を確認します。Pythonの書き方、Flaskの登録機能、認証の役割を分けて読みます。

教材の住所

業務処理： app.py → dashboard 業務処理： app.py → admin_or_guest_required 業務処理： shop_tasks.py → create_shop_tasks_blueprint.list_tasks

property / contextmanager / pytestのmark

これは何か

@propertyはメソッドを属性のように読める形にし、@contextmanagerはwithの流れを作ります。pytestのmarkはテストに条件やパラメータ等を付けます。

Bakery Hubでは

GuestUser.id、_serialize_admin_login_attempt()、test_business_today_uses_jst_at_utc_date_boundary()の@pytest.mark.parametrizeを見比べます。 認証と日付境界のテストの仕組み

なぜ使うか

同じ@でも用途が違うと分かります。日付のテストは1つの検証処理に複数の時刻と期待日を与えます。

教材の住所

業務処理： app.py → GuestUser.id 業務処理： app.py → _serialize_admin_login_attempt テスト： tests/test_business_date.py → test_business_today_uses_jst_at_utc_date_boundary

日付と時間

店舗の今日、保存する時刻、Guestの期限を分けます。

datetime / date / ZoneInfo・日本時間

これは何か

datetimeは日時、dateは日付。ZoneInfoはタイムゾーンを表します。同じ瞬間でも地域によって日付が違います。

Bakery Hubでは

business_today()はAsia/Tokyoの今日を返し、input_sales()はdate.fromisoformat()で送信された日付を変換します。 日本時間の売上入力の仕組み

なぜ使うか

売上入力の今月と本日の金額を日本時間でそろえます。月や年をまたぐと対象商品が変わるため、境界をpytestで確認します。

教材の住所

業務処理： app.py → business_today 業務処理： app.py → input_sales テスト： tests/test_business_date.py → test_business_today_uses_jst_at_utc_date_boundary

実コードの一行 return datetime.datetime.now(BUSINESS_TIMEZONE).date()

UTC / timedelta・期限の比較

これは何か

UTCは時刻の基準、timedeltaは時間差です。日時への加算と比較で有効期限を判断できます。

Bakery Hubでは

utc_now()はUTCの日時を作り、_guest_dataset_is_expired()は絶対期限と最終操作時刻＋無操作期限を現在時刻と比較します。 Guestの利用期限の仕組み

なぜ使うか

店舗の今日という日付と、Guestが何分・何時間使えるかという経過時間を区別し、期限切れの利用を止めます。

教材の住所

業務処理： models.py → utc_now 業務処理： app.py → _guest_dataset_is_expired

PythonとWebアプリの接続

for・if・関数はPythonの基礎。request.formはFlask、queryはSQLAlchemy、fetchはJavaScriptです。Pythonから順に、入力を受ける → DBへ保存する → 画面へ返す、という境界を辿りましょう。

if / for

商品登録 で、不正な値なら拒否し、複数行を順に検証・更新します。条件分岐と繰り返しです。

関数 / 引数 / return

売上分析 の _get_sales_from_db(current_dataset, target_year, target_month) は、対象領域と期間を引数で受け取り、集計結果を返します。

list / dict

売上入力 は検証済みの行をlistにまとめます。 売上分析 は商品名と数量のdictから、グラフ用のlistを作ります。

try / except

売上入力 の日付変換の失敗やDB保存の例外を扱い、エラー応答やrollbackへ進みます。

class / import

models.py の Product(db.Model) などがデータ構造を表すclassです。 app.py は from models import Product などで別ファイルの定義を使います。

GET / POST

GETは画面や集計の取得、POSTは入力送信や操作に使用。 AI取得 も利用回数を消費するためPOSTです。Session復元時のGuest活動日時更新など、GETに伴う共通処理もあります。

request / response

request.args はURLの条件、 request.form はフォーム、 request.get_json() はJSONを読む入口。応答にはHTML、JSON、リダイレクトやエラーのHTTP状態コードがあります。

JSON / GitとDB

商品カタログ のAdmin追加分はJSONと静的画像をGitHubのPRで管理し、merge・Deploy後にテンプレートへ反映します。月次ProductとそのDailySalesはPostgreSQLに保存します。Admin専用POSTでは request.form で商品情報、 request.files で任意画像を受け、認証・認可・検証の後にGitHub APIへ申請します。

Sessionは2つを区別

Flaskの session は 認証状態 を引き継ぐための仕組み。SQLAlchemyの db.session はDB読み書きの作業単位です。Flaskの標準Session Cookieは署名で改ざんを検知しますが、暗号化された秘密の保管庫ではありません。

CRUD

Create（作成）・Read（読み取り）・Update（更新）・Delete（削除）。 店舗メモ で一通り確認できます。商品は履歴を残す販売終了、メモはゴミ箱と完全削除を分けています。

transaction / rollback

関連する変更をひとまとまりで確定・取消する仕組み。 売上入力 では複数行を保存してからcommitし、DB例外ならrollbackします。

validation

入力を保存前に検証すること。 商品登録 の名前・価格や タスク のID検証が例です。HTMLの入力制限に加え、サーバーでも確認します。

template / Jinja

Pythonが渡したデータをHTMLへ埋め込みます。 input.html では商品の繰り返し表示や成功表示に使い、通常の文字列はHTMLエスケープされます。

JavaScript event

タスク の addEventListener() がclick・pointer・touchなどを受けて、画面の変更やサーバーへの送信につなげます。

pytest fixture

conftest.py の flask_app 、 authenticated_client 、 csrf_token が、テスト用DB・認証済みclient・CSRF tokenの準備を共通化します。

共通技術と役割

Python / Flask

入力検証、日付処理、集計結果の整理をPythonで記述。FlaskがURLと関数を結び、HTML・JSON・リダイレクトを返します。店舗ツールはBlueprintで分けています。

SQLAlchemy / PostgreSQL

モデル・query・transactionでデータを扱います。PostgreSQLは本番用DBとDocker / CIのDBに使用。SQLiteはローカルの既定DBと通常pytestに使います。

HTML / Jinja / CSS

HTMLがフォームと見出し、Jinjaがデータの表示、 style.css が配色・カード・PCとスマホの配置を担当。主要メニューは共通テンプレートです。

JavaScript / Chart.js

メニュー開閉、入力操作、JSONの取得と画面更新を担当。ホームは1分ごとに予定と時計を更新し、DashboardのグラフはChart.jsを使います。

Flask-Login / Session

管理者・Guestの認証状態を引き継ぎます。データの利用範囲は別途Datasetで確認します。CookieにはSecure・HttpOnly・SameSiteの設定があり、HTTPのローカル開発時だけSecureを無効にできます。

CSRF / エスケープ

Flask-WTFの CSRFProtect がPOSTのtokenを確認します。JinjaのエスケープやJavaScriptの textContent は、入力文字がHTMLとして実行されることを防ぐために使います。

Gemini API

売上の改善案とひとことを生成。Python側でAPIを呼び、timeout・エラー応答・Guest利用上限を扱います。

pytest / GitHub Actions

pytestが期待する応答とDB状態を検証。 .github/workflows/test.yml は対象パスのmainへのpush・PRで、通常テストとPostgreSQLのmigration・指定された並行処理テストを実行します。

Docker / Gunicorn / Render

DockerfileでPython実行環境を組み立て、GunicornがFlaskアプリへのHTTP要求を処理します。 READMEにRenderでの公開構成を記載。Docker ComposeはwebとPostgreSQLを組み合わせます。

Flask-Migrate / Alembic

migrations/ でDB構造の変更履歴を管理します。 startup_guard.py はコードのmigration headとDBの alembic_version 、必要な表・列を読取専用で照合します。不一致なら通常Web起動を止め、自動で flask db upgrade を実行しません。本番へのmigration適用には別途確認と承認が必要です。

ここではコードと設定ファイルにある設計を説明しています。稼働中の本番環境を診断するページではありません。認証情報や設定の秘密値、実際のCookie・Session値は表示しません。

テストは何を壊さないためにある？

成功画面だけでなく、「拒否されたときにDBが変わっていないか」「他の利用者のデータが混ざっていないか」まで確認します。

入力の境界・不正値 — 商品名・価格・販売数量・メモ本文・タスクタイトルなどの許容範囲を確認します。

認証・権限・CSRF — 未ログインの業務操作、偽装した認証、tokenなしのPOSTを拒否し、公開ページは読めることを確認します。

Dataset分離 — 管理者、Guest A、Guest Bの表示・更新・集計・AI入力が混ざらないことを確認します。

transaction / rollback — 保存失敗で商品・売上・店舗ツールの変更が一部だけ残らないことを確認します。

日付と期間 — 日本時間の今日、月・年の切替、DashboardのHTMLとAPIの期間一致、スケジュールの日付境界を確認します。

CRUD・履歴・並び順 — 販売終了後の売上履歴、メモのゴミ箱と復元、タスクの保存順と一括操作を確認します。

繰り返し・並行処理 — 完了状態の再送、同じ商品・日付の売上更新などを確認。PostgreSQL用にはGuest上限や保存の実並行テストがあります。すべてのUI連打や競合を網羅するという意味ではありません。

表示の安全性 — 入力文字のエスケープ、URL処理、セキュリティヘッダー、共通navigationを確認します。

DB構造の変更 — migrationのupgrade / downgrade、既存データの保全、DB制約を確認します。

通常テストは conftest.py の隔離DBを使います。PostgreSQL専用テストは使い捨てのテストDBと明示的な設定が必要です。AI連携テストはクライアントを置き換えます。pytestのGREENだけでは、スマホの操作感や本番環境での動作をすべて保証できないため、ブラウザでの表示確認も組み合わせます。

conftest.py

tests/test_csrf.py

tests/test_xss_regressions.py

tests/test_system_overview.py

困ったときはどこを見る？

症状に近いカードを開き、最初の確認場所を探しましょう。ブラウザの開発者ツールの「Network」では、送ったURL・HTTPメソッド・応答の状態を確認できます。

ボタンを押しても何も起きない

最初に： HTML form / JavaScript / route

Networkに送信がなければ、フォームの入力制限やJavaScriptのConsoleエラーを確認。送信があればURL・メソッドとFlaskのrouteを見比べます。

売上入力 の templates/input.html から確認できます。

POSTで400になる

最初に： 応答本文 / CSRF token / 入力検証

このアプリのCSRF拒否は400です。formの csrf_token やfetchの X-CSRFToken を確認します。入力検証も400を返すため、応答本文からどちらかを見分けます。

認証とフォーム 、 tests/test_csrf.py が手掛かりです。

入力した内容が保存されない

最初に： HTTP応答 / validation / transaction

400なら検証で保存前に終了した可能性があります。500ならDB障害とrollbackを確認。送信した日付と商品IDも照合し、成功表示だけで判断せず保存対象を追います。

売上入力 の input_sales() と tests/test_sales.py を読みます。

DBにはあるのに画面に出ない

最初に： queryの条件 / template / JSON

Dataset・年月・販売状態で対象外になっていないか確認します。Dashboardなら表示期間とAPIのJSONを比べ、データ取得と画面更新のどちらで抜けたか絞ります。

売上分析 の _get_sales_from_db() と templates/dashboard.html が入口です。

日付や月の切り替わりがおかしい

最初に： 日本時間の今日 / 売上日 / 表示期間

business_today() は日本時間の今日を返します。入力画面は今月の商品、Dashboardの本日売上は今日、ランキングは選択期間を見るので、どの日付を使う処理か区別します。

売上入力 と tests/test_business_date.py で境界を確認します。

Guestのデータが混ざって見える

最初に： 認証されたDataset / queryの絞り込み

require_current_dataset() の結果を使っているか、queryやjoinでDataset条件が抜けていないか確認します。URLやformのDataset IDを認可の根拠にすると越境の原因になります。

Guest Demo と tests/test_dataset_authorization.py で利用者の境界を確認します。

並び順がおかしい

最初に： 保存された順番 / query / Python / JavaScript

Dashboardは sorted() で数量順、タスクは position で順番を管理します。まずサーバーが返す順番を見て、正しければJavaScriptの表示・並び替え処理を確認します。

タスク の shop_tasks.py と static/shop_tasks.js を見比べます。

pytestだけ失敗する

最初に： fixture / テストDB / assertion / 日付

conftest.py のfixtureが準備した認証・商品年月と、期待値を照合します。通常テストはメモリ内SQLiteなので、本番のデータがある前提では動きません。日付依存やDBの違いも確認します。

売上入力のテスト を入口に、失敗したassertionが守る条件を読みます。